

Analisi e Progettazione del Software

Pattern GRASP

Oliviero Riganelli

Progettazione orientata agli oggetti

- Progettazione orientata agli oggetti
 - *dopo aver identificato i requisiti e aver creato un modello di dominio,*
 - *si aggiungano i metodi alle classi appropriate,*
 - *e si definiscano i messaggi tra gli oggetti per soddisfare i requisiti*
- La padronanza della progettazione orientata agli oggetti coinvolge un insieme ampio di principi flessibili

UML e i principi di progettazione

Lo strumento critico della progettazione per lo sviluppo software è una mente ben istruita sui principi di progettazione

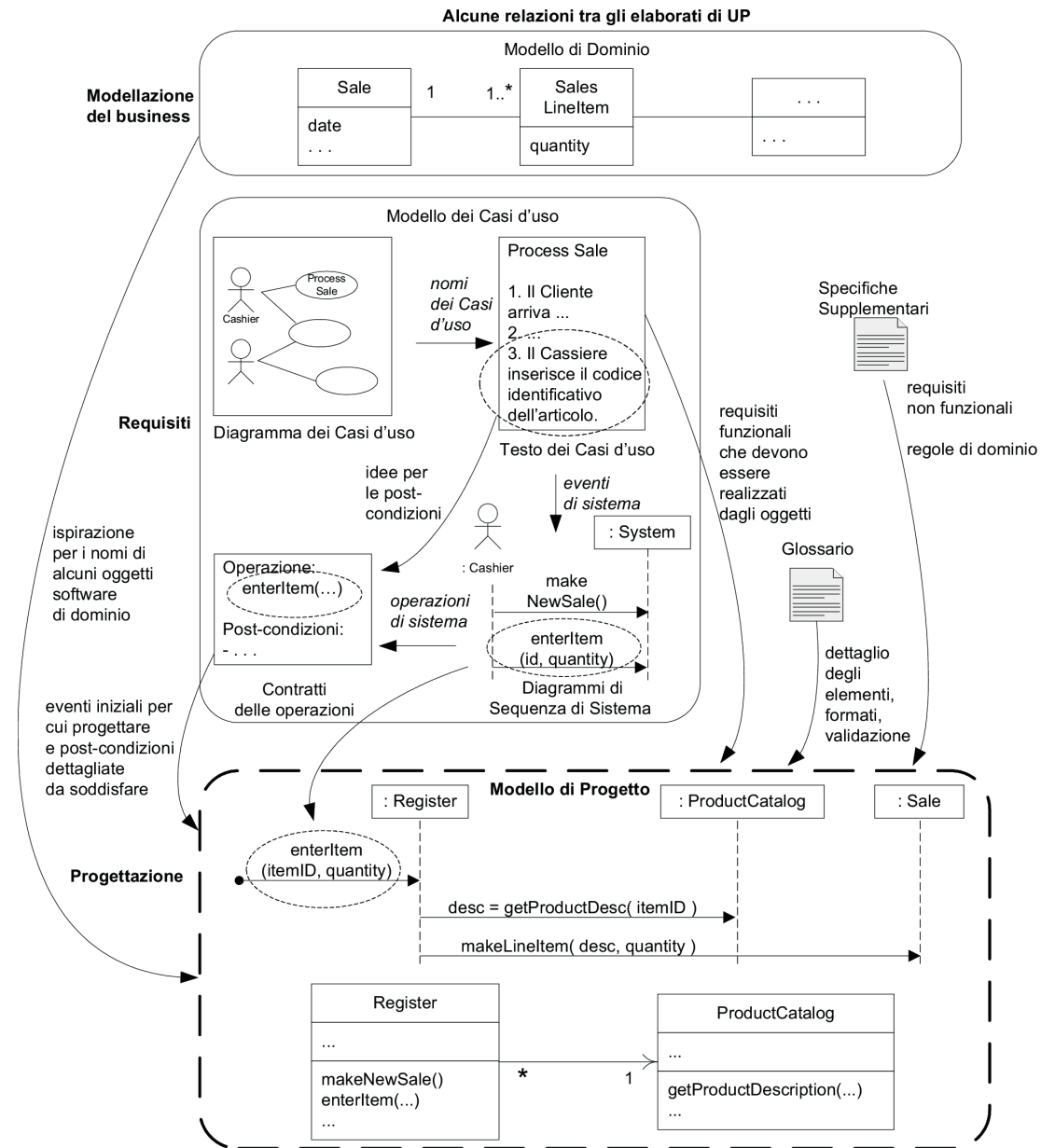
- Non è UML che è solo una notazione
- Non è qualsiasi altra tecnologia

Quali sono gli input della progettazione a oggetti

- testo dei casi d'uso
- specifiche supplementari
- glossario
- diagrammi di sequenza di sistema
- modello di dominio
- contratti delle operazioni

Relazione tra gli elaborati

Con enfasi sull'influenza sulla progettazione orientata agli oggetti



Quali sono le attività della progettazione a oggetti

- Dato uno o più di questi input si può
 - Progettare mentre si codifica, idealmente con lo **sviluppo preceduto dai test**
 - Iniziare con un po di modellazione UML per la progettazione a oggetti
 - Iniziare con un'altra tecnica di modellazione, come le schede CRC
- La cosa più importante è che vengano applicati i vari principi di progettazione orientata agli oggetti
 - */ pattern GRASP*
 - */ design pattern “Gang-of-Four”*
 - *Progettazione guidata dalle responsabilità*
- Disegnare i modelli soprattutto allo scopo di **comprendere e comunicare**

Quali sono gli output della progettazione a oggetti

- Diagrammi UML di interazione, delle classi e dei package
- Abbozzi e prototipi dell'interfaccia utente
- Modelli delle base di dati

Responsabilità e progettazione guidata dalle responsabilità

- **Progettazione guidata dalle responsabilità (RDD)**
 - *Pensare in termini di **responsabilità**, **ruoli** e **collaborazioni***
 - *Per responsabilità si intende un'astrazione di ciò che fa o rappresenta un oggetto*
- UML definisce una responsabilità come “*un contratto o un obbligo di un classificatore*”

Responsabilità

Le responsabilità sono fondamentalmente di due tipi:

- Le responsabilità **di fare** di un oggetto comprendono:
 - *Fare qualcosa esso stesso, come per esempio eseguire un calcolo*
 - *Chiedere ad altri oggetti di eseguire azioni*
 - *Controllare e coordinare le attività di altri oggetti*
- Le responsabilità **di conoscere** di un oggetto comprendono
 - *Conoscere i propri dati privati incapsulati*
 - *Conoscere gli oggetti correlati*
 - *Conoscere cose che può derivare o calcolare*

Responsabilità

- Le responsabilità sono assegnate alle classi durante la progettazione di oggetti. Per esempio, possiamo dichiarare quanto segue:
 - *"una Sale è responsabile della creazione di SalesLineItems" (fare)*
 - *"una Sale è responsabile di conoscere il suo totale" (sapere)*
- Le responsabilità relative alla "conoscenza" sono spesso deducibili dal Modello di Dominio (per gli attributi e le associazioni che mostra)

Responsabilità e metodi

- La traduzione delle responsabilità in classi e metodi è influenzata dalla granularità della responsabilità.
 - *Ad esempio, "fornire l'accesso a banche dati relazionali" può comportare decine di classi e centinaia di metodi, mentre "creare una Sale" può comportare solo uno o pochi metodi.*
- Una responsabilità non è la stessa cosa di un metodo, ma i metodi sono implementati per adempiere alle responsabilità.
- I metodi agiscono da soli o collaborano con altri metodi e oggetti.

Responsabilità e collaborazioni

- Le responsabilità sono implementate per mezzo di oggetti e metodi che
 - *agiscono da soli*
 - *oppure che **collaborano** con altri oggetti e metodi*
- La progettazione guidata dalle responsabilità porta a considerare un progetto OO come *una comunità di oggetti con responsabilità che collaborano*

Flusso di lavoro RDD

La progettazione guidata dalle responsabilità viene fatta iterativamente come segue:

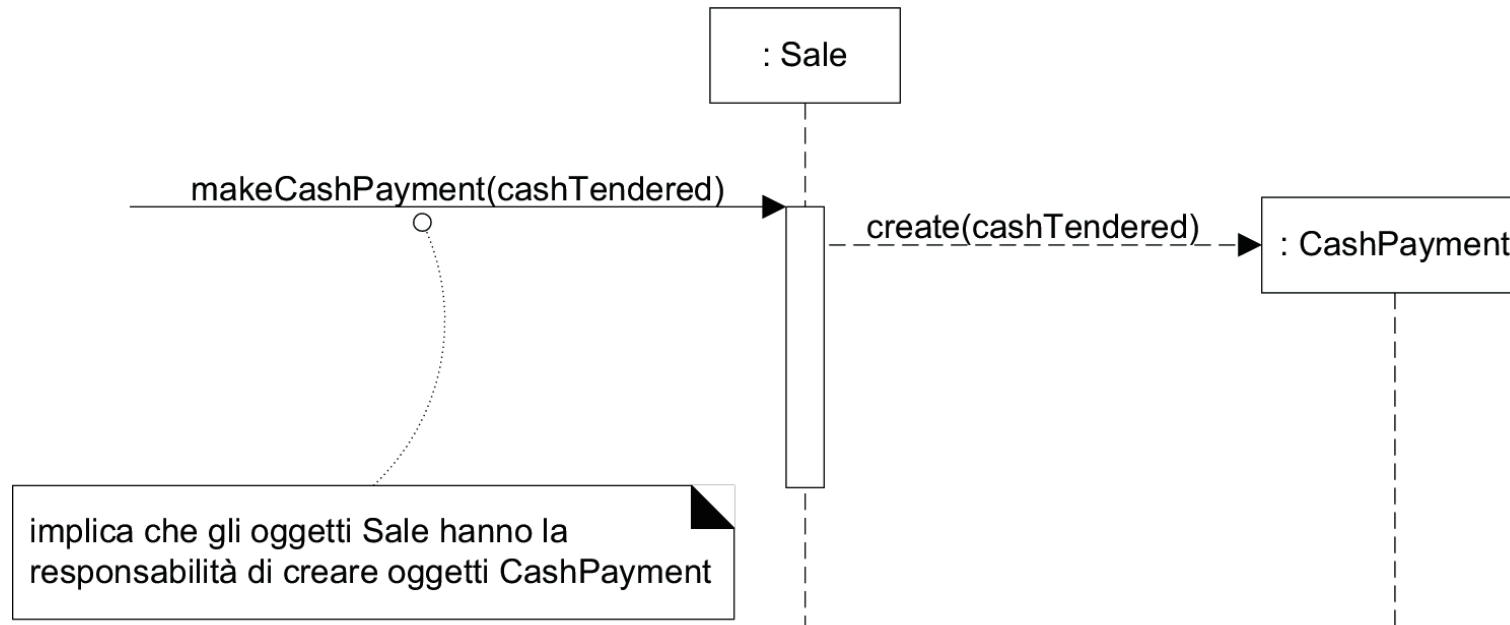
- Identifica le responsabilità, e considerale una alla volta
- Chiediti a quale oggetto software assegnare questa responsabilità (potrebbe essere un oggetto già identificato o nuovo)
- Chiediti come fa l'oggetto scelto a soddisfare questa responsabilità (da solo o collaborando)

Pattern GRASP

- I pattern GRASP danno un nome e descrivono principi di base per la progettazione di oggetti e l'assegnazione di responsabilità
 - **General Responsibility Assignment Software Patterns**
- Pattern GRASP di base
 - *Creator*
 - *Information Expert*
 - *Low Coupling*
 - *High Cohesion*
 - *Controller*
- Altri pattern GRASP
 - *Polymorphism, Pure Fabrication, Indirection, Protected Variations*

Qual è la connessione tra responsabilità, GRASP e diagrammi UML

- Nell'ambito di UML, il disegnare i diagrammi di interazione diventa l'occasione per considerare tali responsabilità (realizzate come metodi)
- *Mentre si disegna un diagramma di interazione vanno prese delle decisioni riguardo all'assegnazione di responsabilità*



Che cosa sono i pattern

- I progettisti OO esperti basano la progettazione su un insieme di principi generali e soluzioni idiomatiche

Nome del pattern:	Information Expert
Problema:	Qual è un principio di base con cui assegnare responsabilità agli oggetti?
Soluzione:	Assegna una responsabilità alla classe che ha le informazioni necessarie per soddisfarla.

Che cosa sono i pattern

- Un buon pattern è una coppia problema/soluzione ben conosciuta e con un nome
 - *può essere applicato in nuovi contesti*
 - *contiene consigli su come applicarla in situazioni nuove*
 - *discute compromessi, implementazioni, variazioni...*
- I pattern codificano soluzioni, idiomi e principi già applicati e che si sono dimostrati corrette in varie situazioni
 - *i pattern non affermano nuove idee*
 - *i pattern codificano dei principi di base ampiamente utilizzati*
- I pattern hanno un nome
 - *facilitano comprensione, memorizzazione e comunicazione*

A che punto siamo?

1. Siamo nel contesto di un processo iterativo, abbiamo svolto alcune attività e prodotto alcuni elaborati.
2. La RDD come metafora per la progettazione degli oggetti
3. I pattern come modo per dare un nome e spiegare le idee della progettazione OO
4. UML per la modellazione visuale per la progettazione OO, nel corso della quale possono essere applicati sia i pattern GRASP che quelli GoF

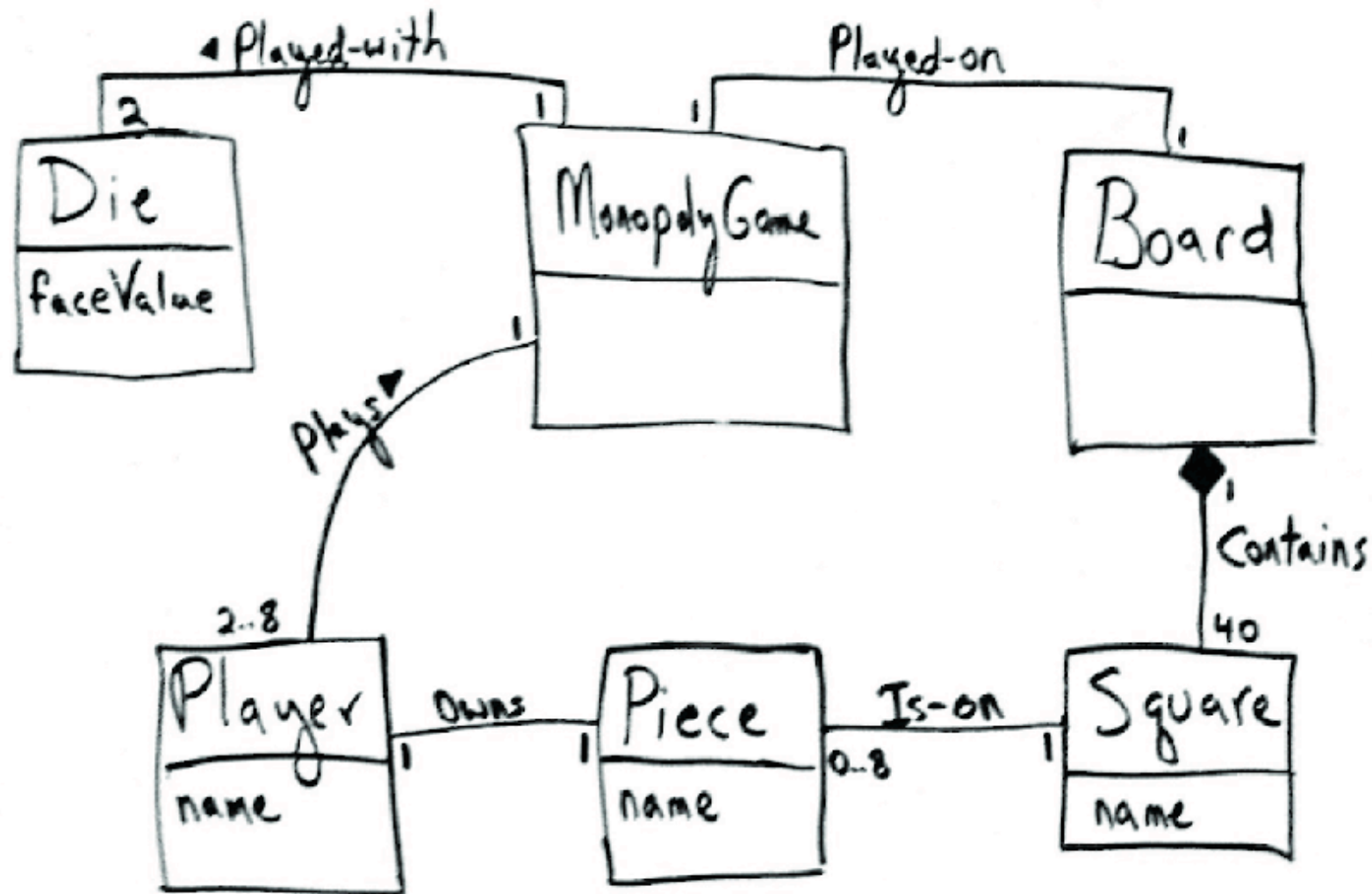
Problema: Chi crea gli oggetti Square?

- Responsabilità di fare
 - *creare oggetti di tipo Square*
 - *più in generale – creare oggetti di tipo X*
- Una risposta sbagliata
 - *gli oggetti Square sono creati da un oggetto Dog (ovvero una classe arbitraria)*
 - *perché sbagliata?*

Pattern Creator

Nome:	Creator (Creatore)
Problema:	Chi crea un oggetto A?
Soluzione (può essere vista come un consiglio):	Assegna alla classe B la responsabilità di creare un'istanza della classe A se una delle seguenti condizioni è vera (più sono vere, meglio è): • B “contiene” o aggrega con una composizione oggetti di tipo A. • B registra A. • B utilizza strettamente A. • B possiede i dati per l'inizializzazione di A.

Modello di dominio per Monopoly



Board creerà gli oggetti Square

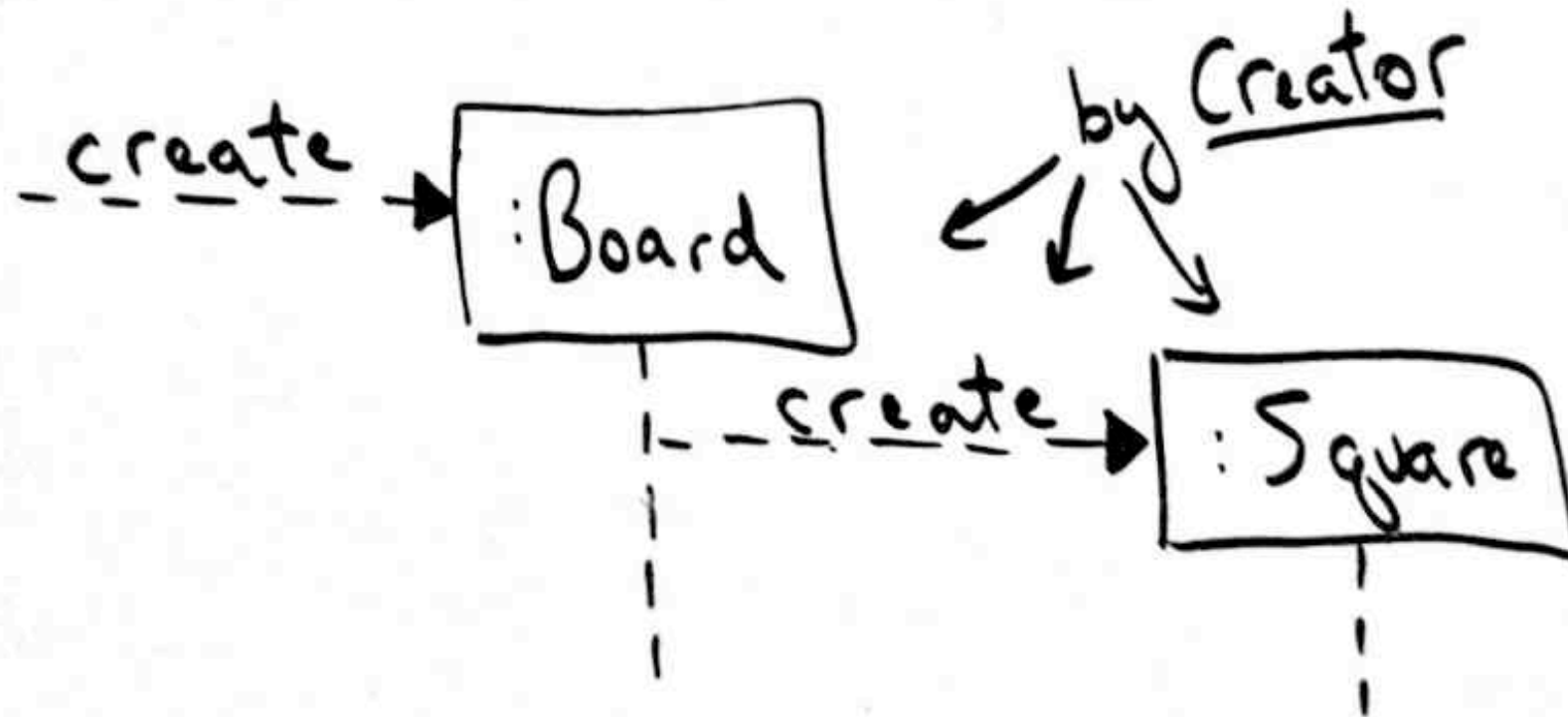
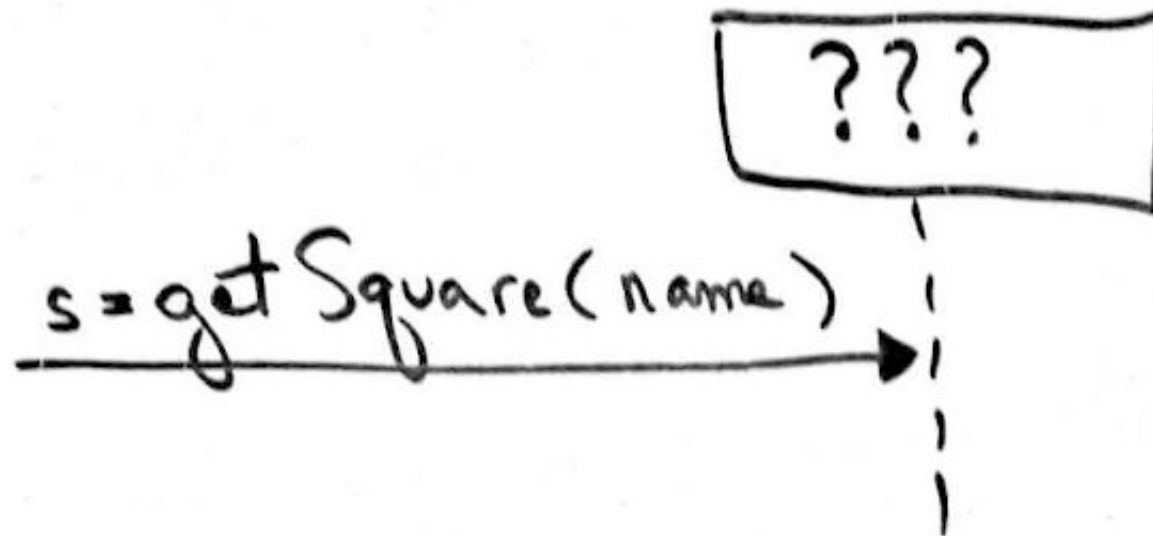


Diagramma delle classi di progetto



Problema: Chi conosce gli oggetti Square?

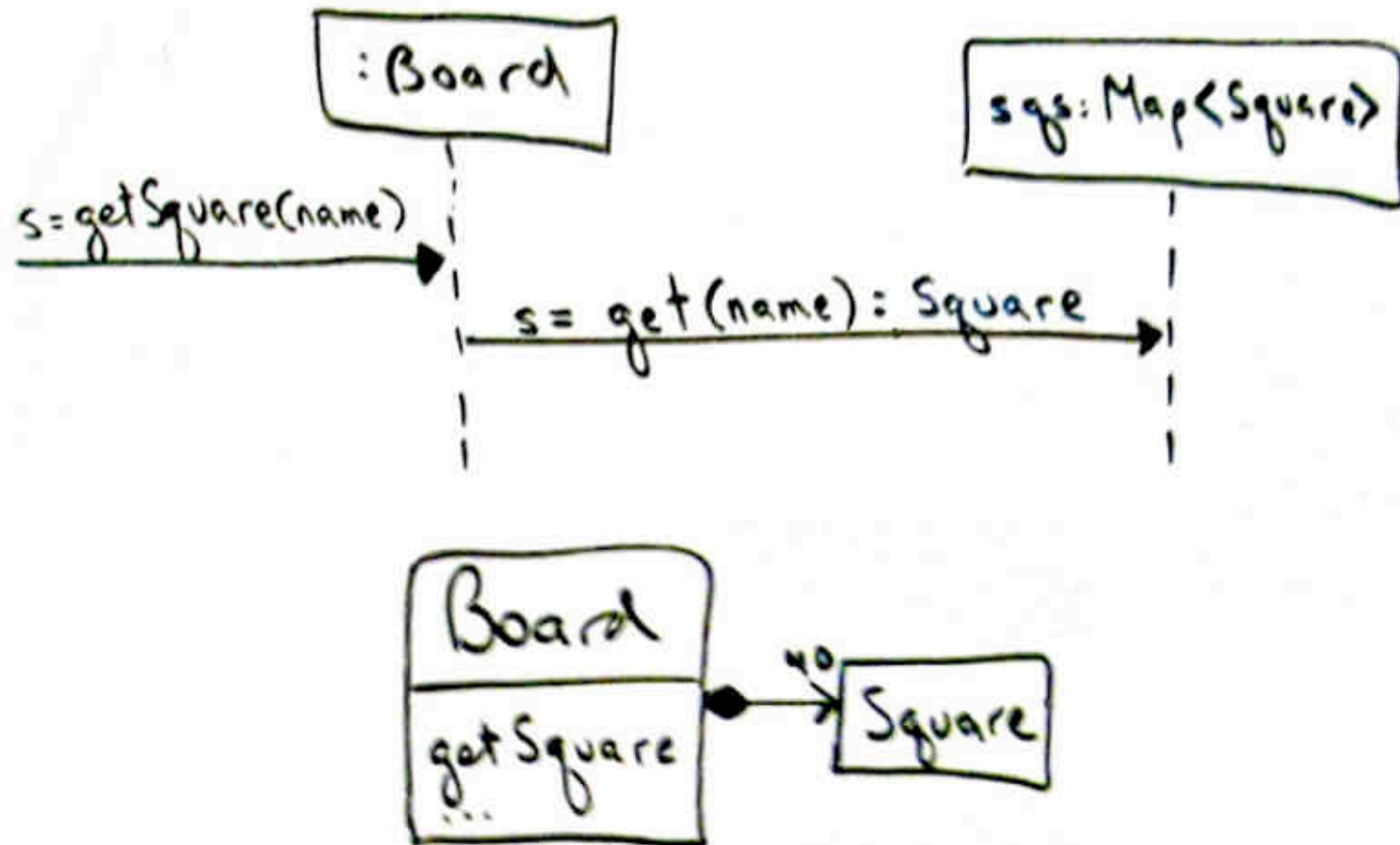
- Responsabilità di conoscere
 - *trovare un oggetto Square, dato il suo nome univoco*



Information Expert

Nome:	Information Expert (Esperto delle Informazioni)
Problema:	Qual è un principio di base per assegnare responsabilità agli oggetti?
Soluzione (consiglio):	Assegna una responsabilità alla classe che possiede le informazioni necessarie per soddisfarla.

Board possiede le informazioni necessarie per soddisfare questa responsabilità



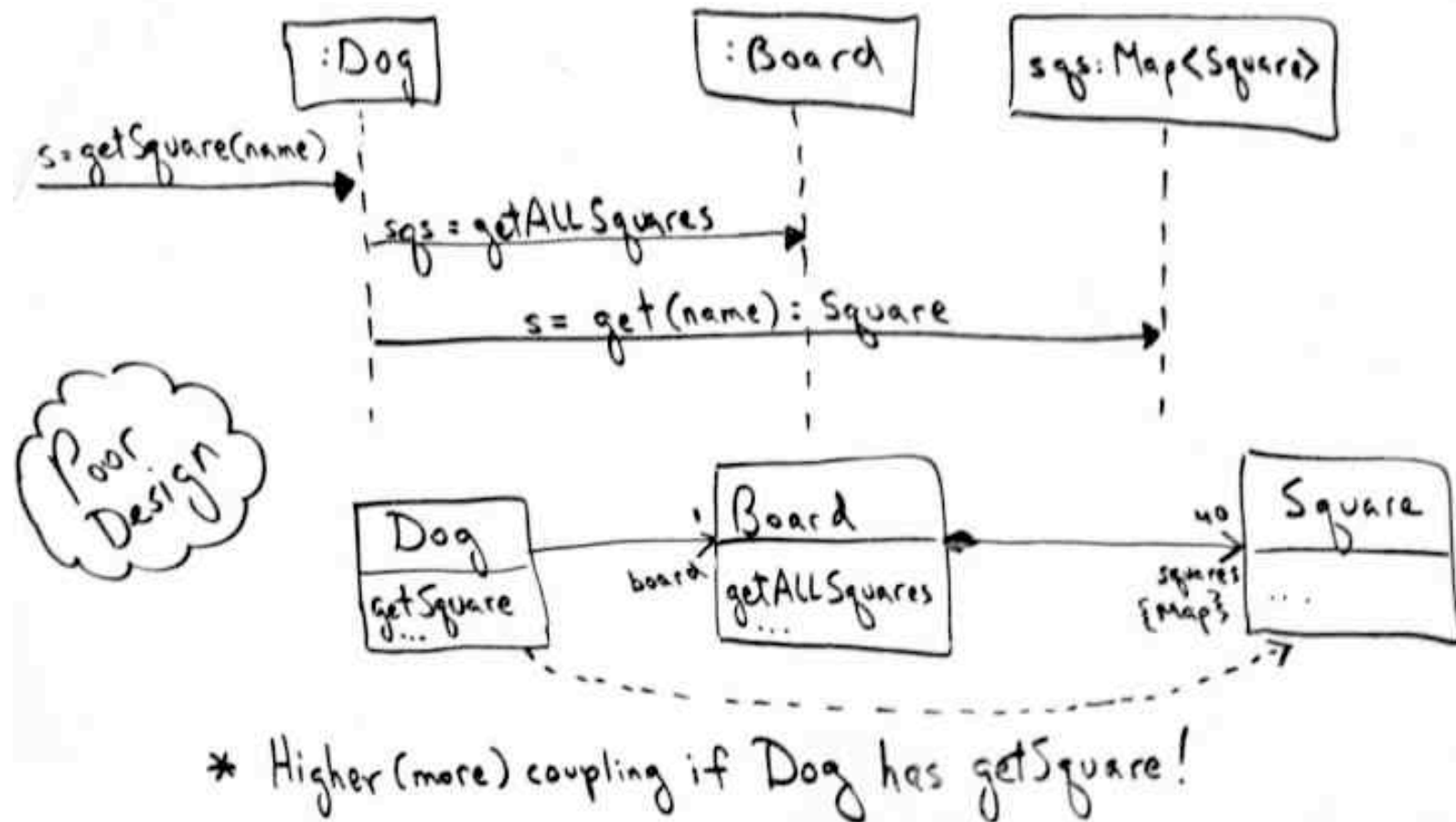
Perché Board anziché Dog?

- Perché la responsabilità di cercare oggetti Square è stata data a Board e non a Dog?
- Accoppiamento
 - *una misura di quanto fortemente un elemento è connesso ad altri elementi, li conosce, o dipende da essi*
- Un principio fondamentale di progettazione
 - assegna le responsabilità in modo che l'accoppiamento (inutile) rimanga basso

Low Coupling

Nome:	Low Coupling (Accoppiamento Basso)
Problema:	Come ridurre l'impatto dei cambiamenti?
Soluzione (consiglio):	Assegna le responsabilità in modo tale che l'accoppiamento (non necessario) rimanga basso. Usa questo principio per valutare le alternative.

Valutazione dell'effetto dell'accoppiamento in questo progetto



Information Expert sostiene Low Coupling

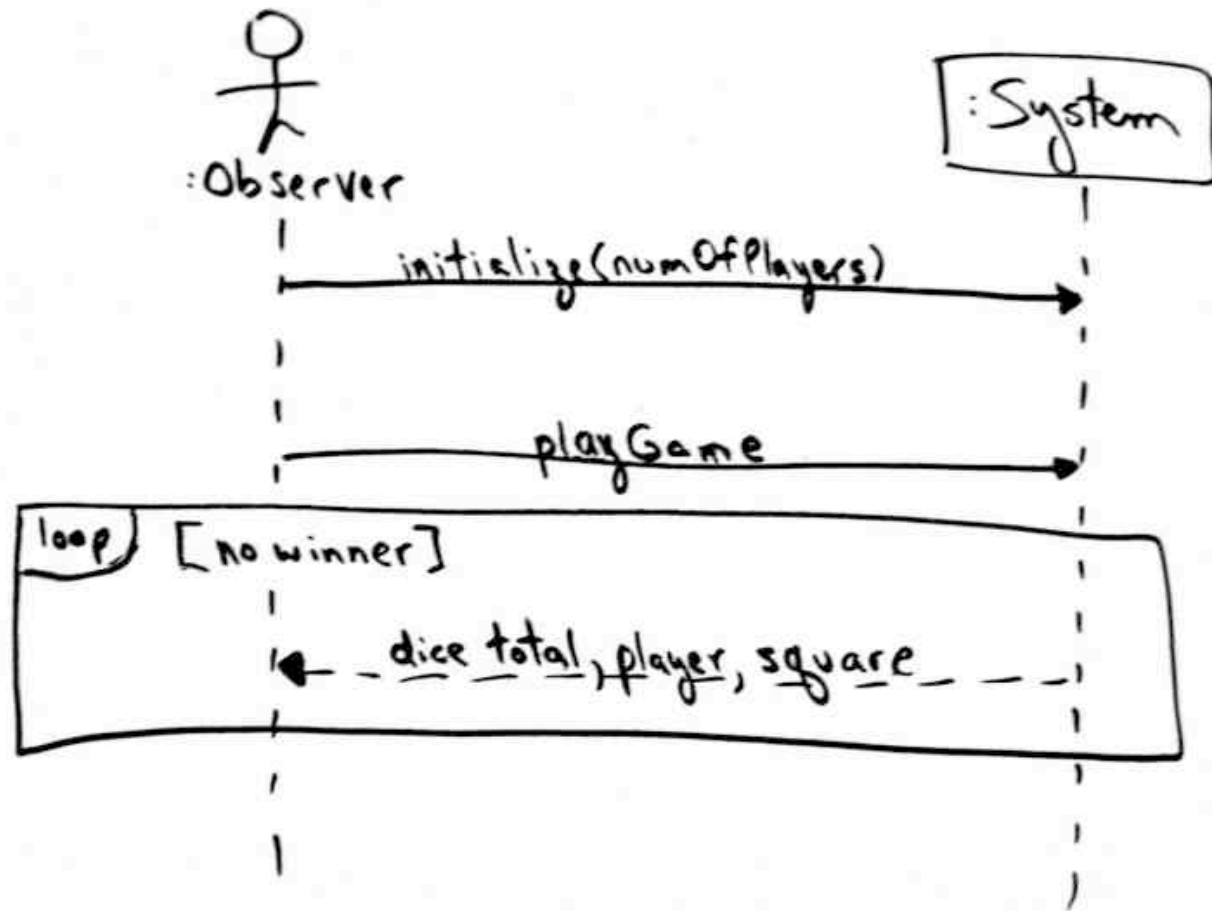
Information Expert guida verso una scelta che sostiene Low Coupling

- Expert chiede di trovare l'oggetto che possiede la maggior parte delle informazioni richieste per la responsabilità
- se ignoriamo i consigli di Information Expert, l'accoppiamento complessivo è maggiore
 - Ci sono più oggetti o informazioni che devono essere condivisi

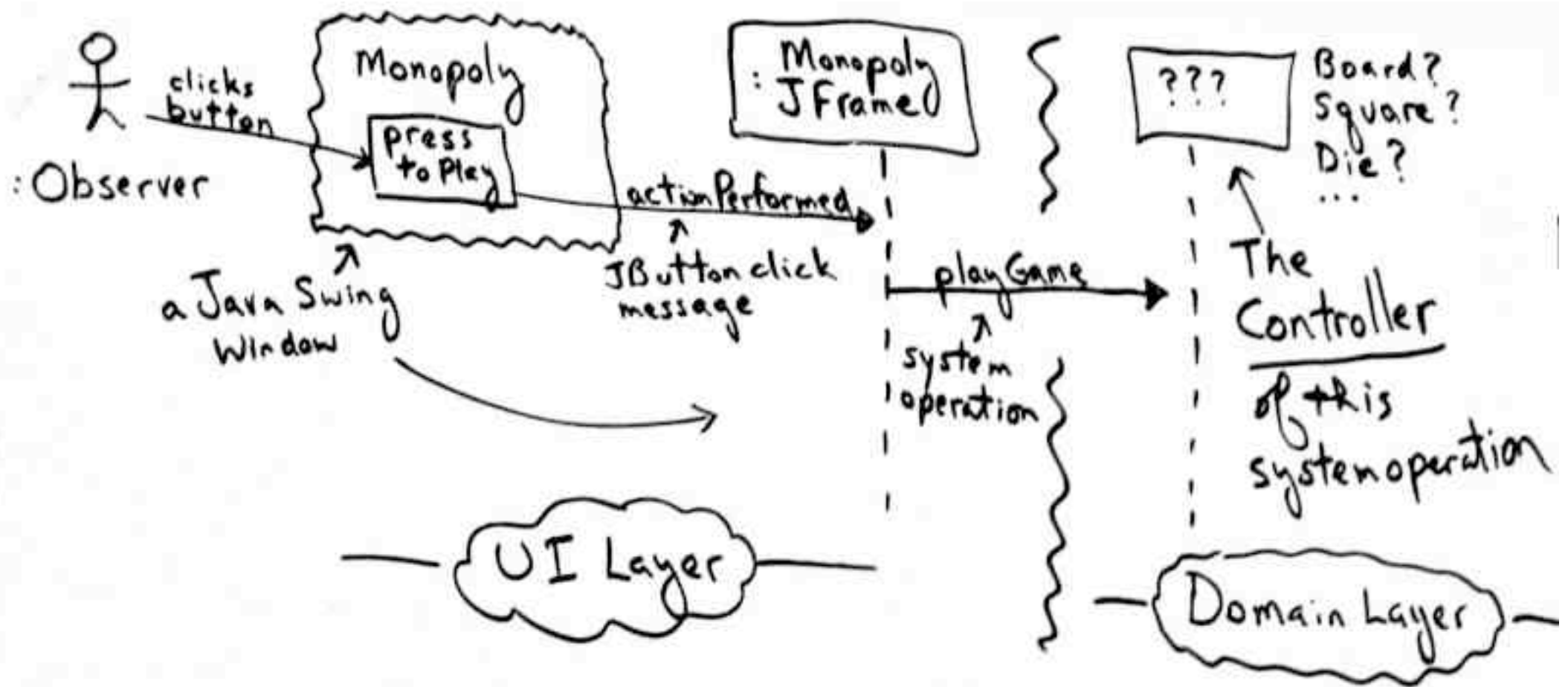
Problema: Quale è il primo oggetto dopo o oltre lo strato di UI che riceve il messaggio dallo strato UI?

- Responsabilità
 - *l'utente del sistema genera eventi, catturati dalla UI*
 - *la UI **delega** allo strato del dominio l'esecuzione delle operazioni di sistema*
 - *quale oggetto dello strato non-UI riceve richieste direttamente dalla UI?*

Quale è il primo oggetto dopo o oltre lo strato di UI che riceve il messaggio dallo strato UI?



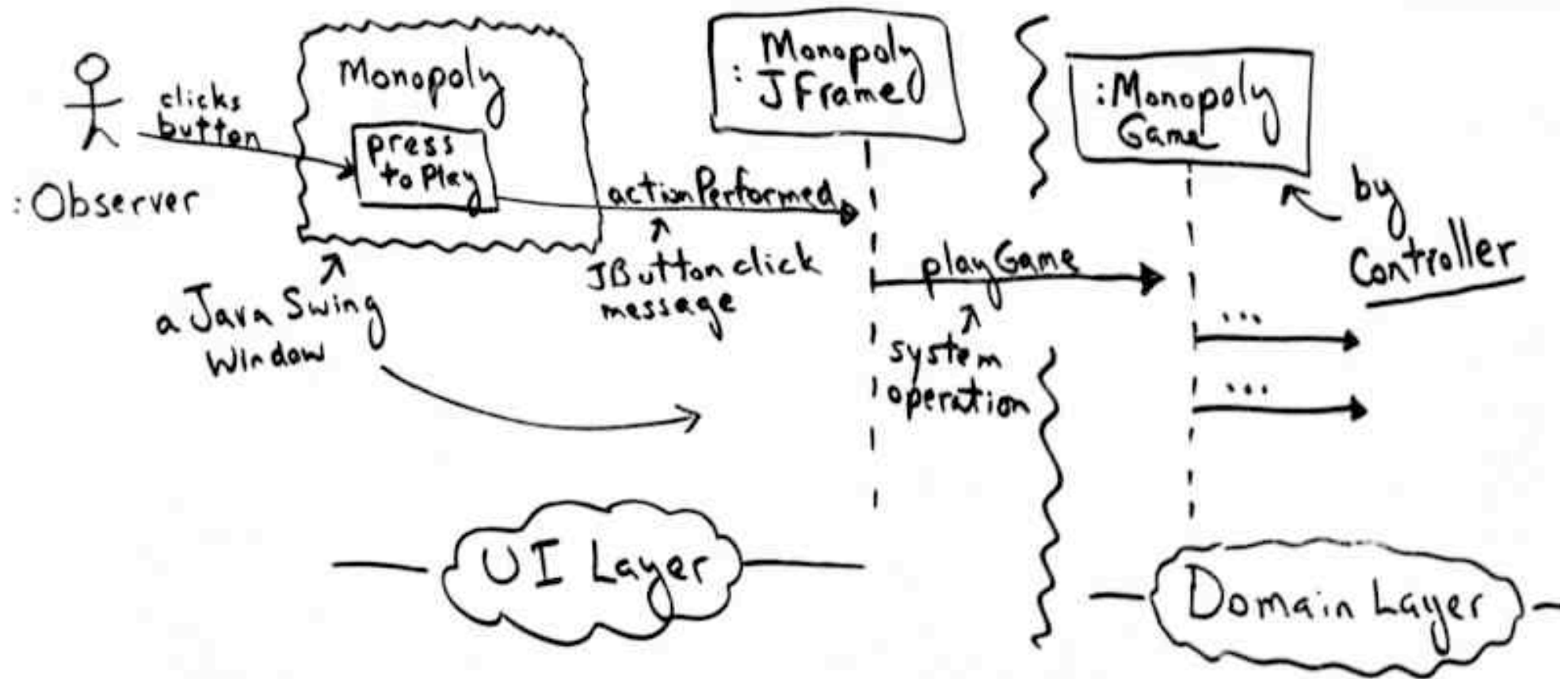
Quale è il primo oggetto dopo o oltre lo strato di UI che riceve il messaggio dallo strato UI?



Controller

Nome:	Controller (Controllore)
Problema:	Qual è il primo oggetto oltre lo strato UI a ricevere e coordinare (“controllare”) un’operazione di sistema?
Soluzione (consiglio):	Assegna la responsabilità a un oggetto che rappresenta una delle seguenti scelte. • Rappresenta il “sistema” complessivo, un “oggetto radice”, un dispositivo all’interno del quale viene eseguito il software, un punto d’accesso al software o un sottosistema principale (sono tutte varianti di un <i>facade controller</i>). • Rappresenta uno scenario di un caso d’uso all’interno del quale si verifica l’operazione di sistema (un <i>controller di caso d’uso</i> o <i>controller di sessione</i>)

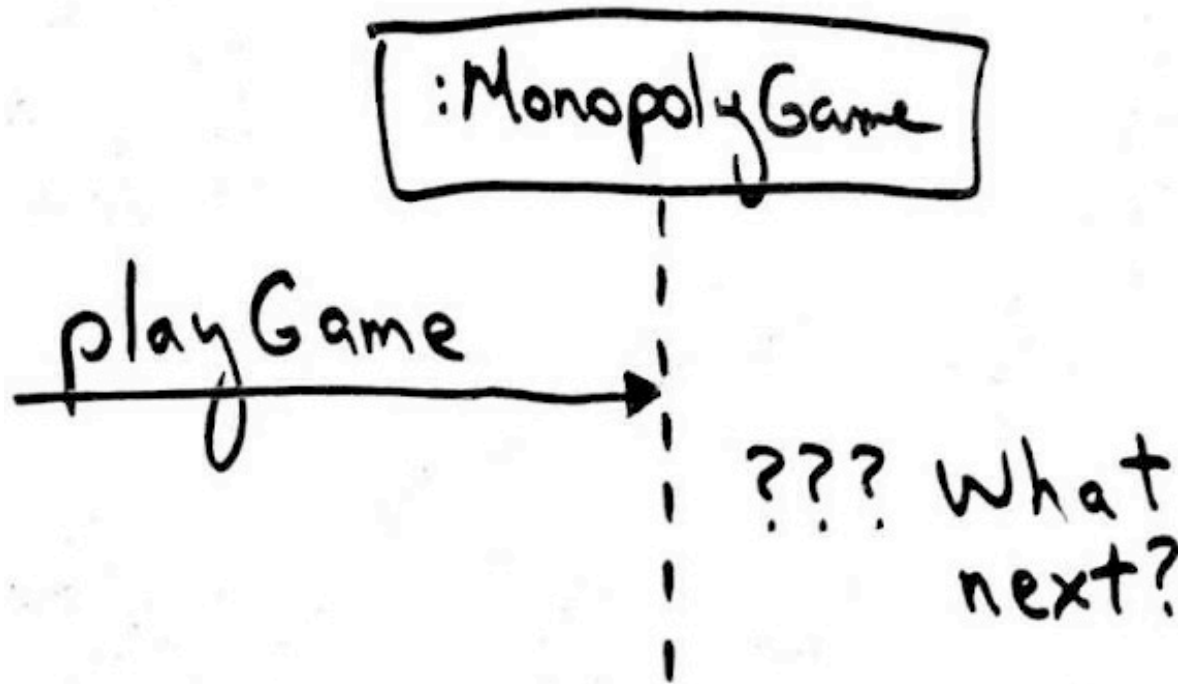
MonopolyGame riceve richieste dalla UI



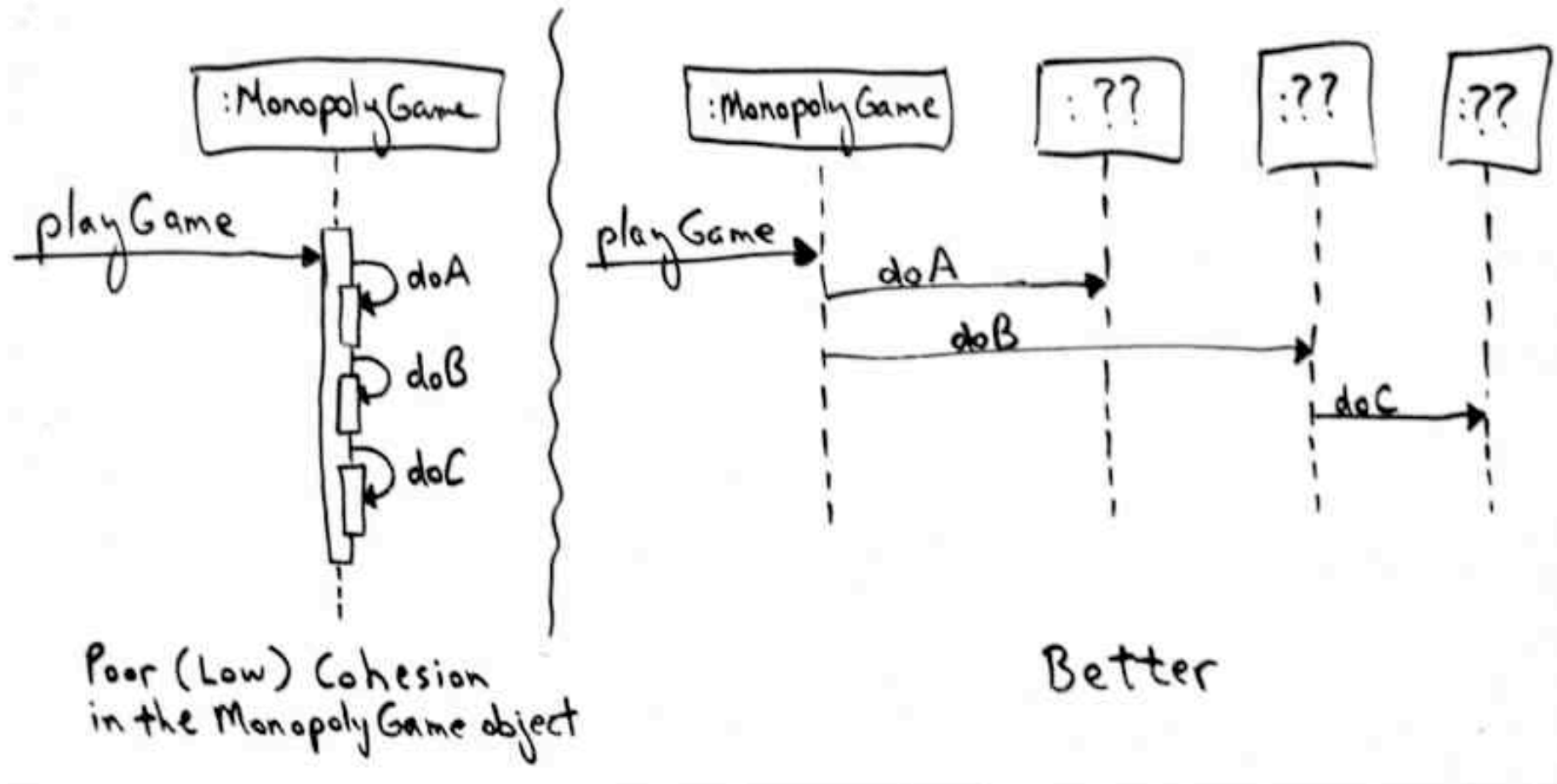
Casa succede dopo?

Controller suggerisce di assegnare la responsabilità di *playGame* a *MonopolyGame*

- come fa *MonopolyGame* a soddisfare questa responsabilità?



Due approcci contrapposti



Coesione

- una misura di quanto sono correlate le operazioni di un elemento software da un punto di vista funzionale
- Un principio fondamentale di progettazione
 - assegna le responsabilità in modo che la coesione rimanga alta

High Cohesion

Nome:	High Cohesion (Coesione Alta)
Problema:	Come mantenere gli oggetti focalizzati, comprensibili e gestibili e, come effetto collaterale, sostenere Low Coupling?
Soluzione (consiglio):	Assegna le responsabilità in modo tale che la coesione rimanga alta. Usa questo principio per valutare le alternative.

Applicare i pattern GRASP alla progettazione a oggetti

- Pattern GRASP
 - *General Responsibility Assignment Software Patterns*
- Pattern GRASP di base
 - *Creator*
 - *Information Expert*
 - *Low Coupling*
 - *High Cohesion*
 - *Controller*
- Altri pattern GRASP
 - *Polymorphism, Pure Fabrication, Indirection, Protected Variations*

Creator

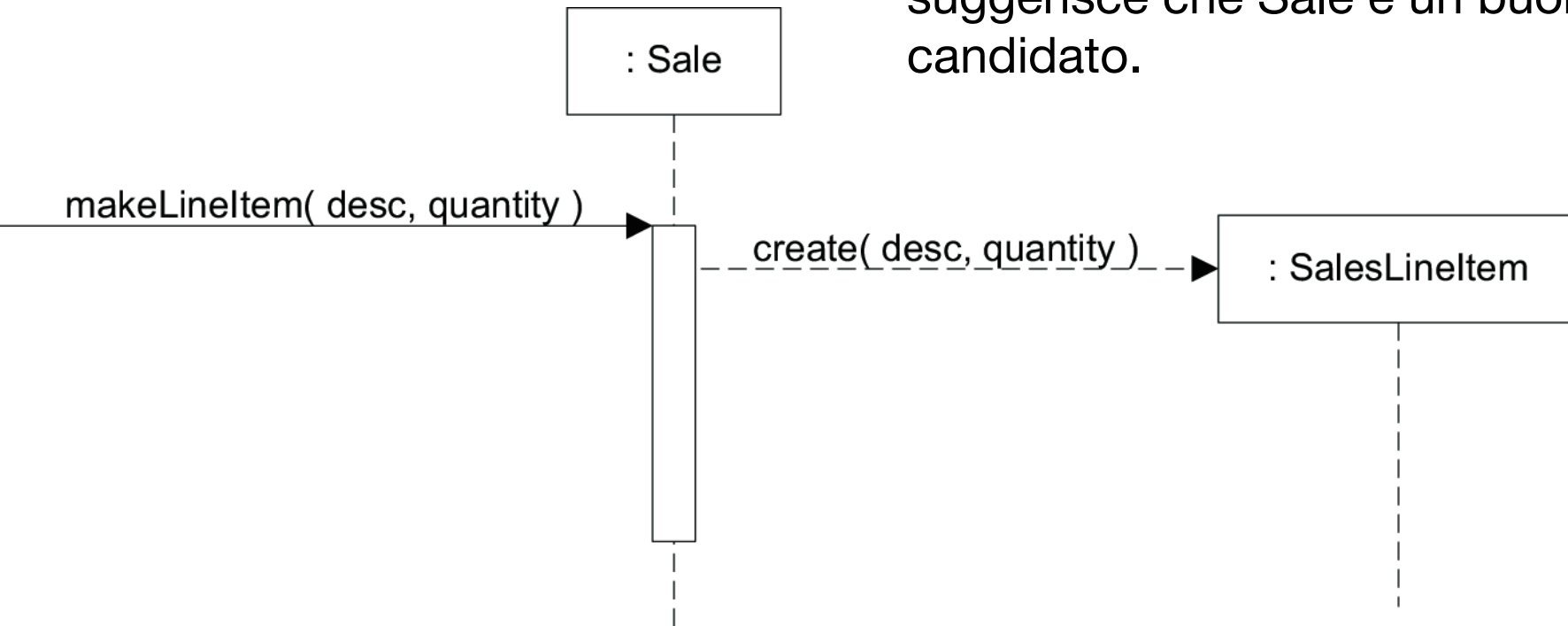
- **Problema:** Chi dovrebbe essere responsabile della creazione di una nuova istanza di una classe?
- La creazione di oggetti è una attività necessaria e comune nel software orientato agli oggetti
 - *utile avere un principio generale per assegnare questo tipo di responsabilità*
 - *Se questo vengono assegnate bene, il progetto può sostenere un accoppiamento basso, una maggiore chiarezza, l'incapsulamento e la riusabilità*

Creator

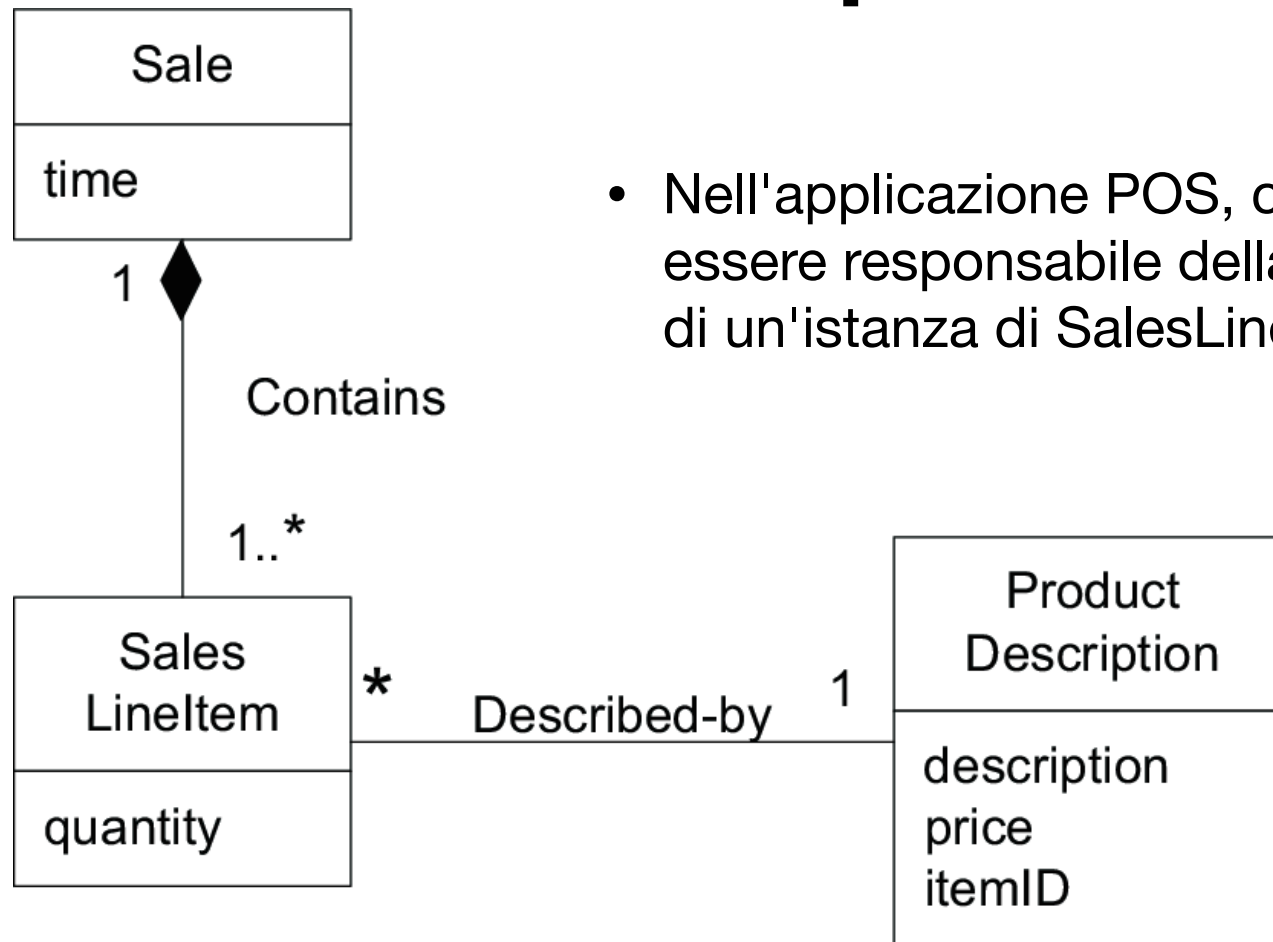
- **Soluzione:** assegnare alla classe B la responsabilità di creare un'istanza di A se una o più delle seguenti condizioni è vera:
 - B *aggrega* gli oggetti A.
 - B *contiene* gli oggetti A.
 - B *registra* le istanze degli oggetti A.
 - B *ha i dati di inizializzazione* che saranno passati ad A al momento della creazione (quindi B è un Esperto rispetto alla creazione di A).

Esempio

- Poiché Sale contiene molti oggetti SalesLineItem, il pattern Creator suggerisce che Sale è un buon candidato.



Esempio



- Nell'applicazione POS, chi dovrebbe essere responsabile della creazione di un'istanza di SalesLineItem?

Discussione

- Creator guida l'assegnazione di responsabilità relative alla creazione di oggetti
 - *Lo scopo è trovare un creatore che abbia bisogno di essere connesso all'oggetto creato (favorisce un accoppiamento basso)*
- Creator suggerisce che la classe per il “contenitore che racchiude” o per il “registratore” siano buoni candidati per la responsabilità di creare la cosa contenuta o registrata
- Si noti che è stato utilizzato il concetto di composizione tra le considerazioni per applicare il pattern Creator (un oggetto composto è un ottimo candidato per creare le sue parti)
 - utile mostrare la composizione nel modello di dominio

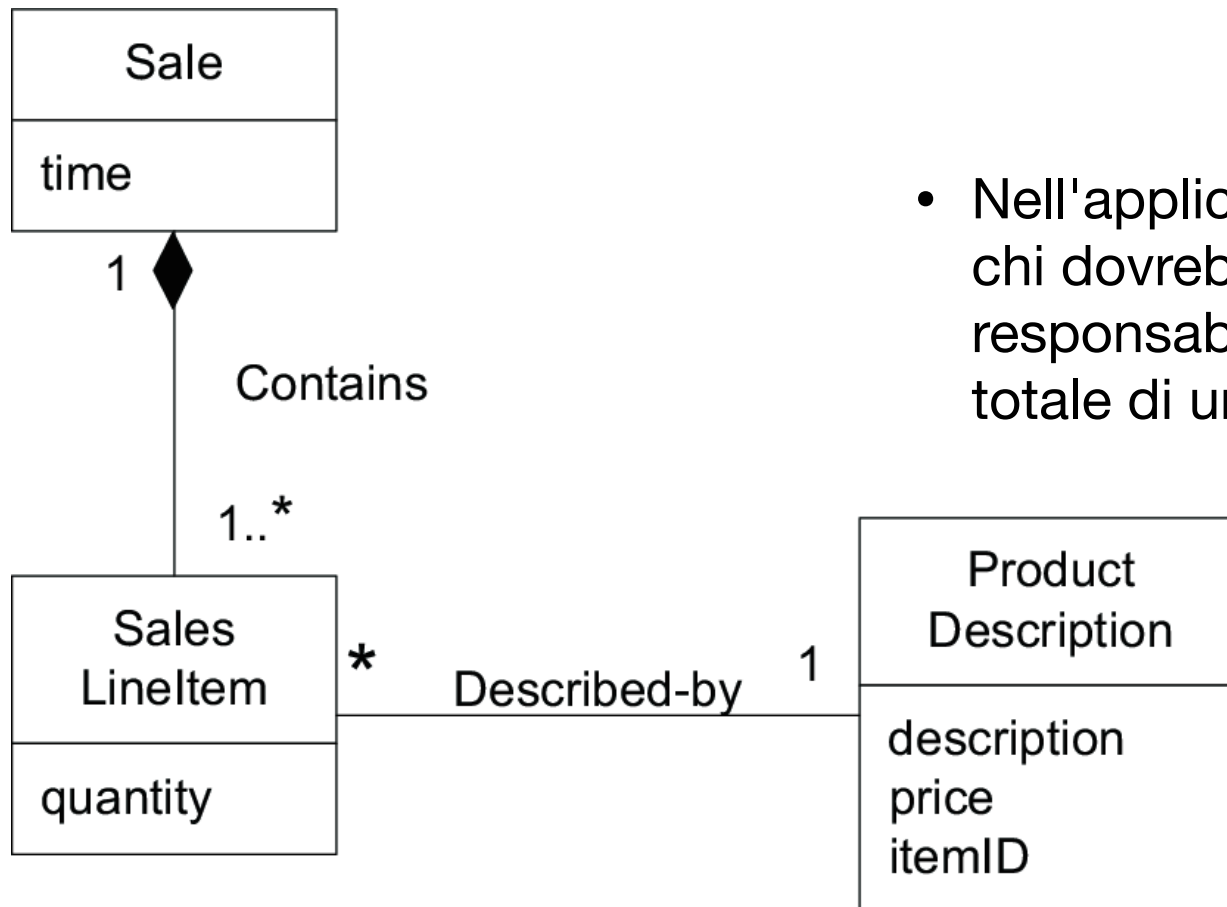
Controindicazioni, vantaggi e pattern correlati

- Controindicazioni
 - *se la creazione dell'oggetto è complessa – usa altre soluzioni ad es., Abstract Factory [GoF]*
- Vantaggi
 - *favorisce un accoppiamento basso, poiché gli oggetti creati sono probabilmente già visibili all'oggetto creatore*
- Pattern principi correlati
 - *Low Coupling,*
 - *I design pattern creazionali, come Abstract Factory*
 - *Whole-Part descrive un pattern per definire oggetti aggregati che consente l'incapsulamento dei componenti*

Information Expert (o Expert)

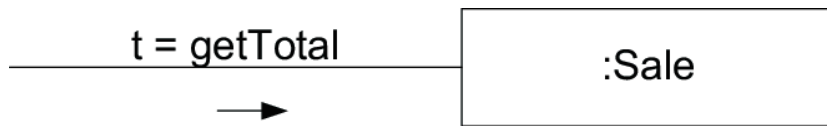
- **Problema:** qual è il principio generale dell'attribuzione delle responsabilità agli oggetti?
- Durante la progettazione si effettuano delle scelte sull'assegnazione delle responsabilità a centinaia o migliaia di classi software
 - *utile un principio generale, per l'assegnazione delle responsabilità più comuni*
 - *Se le scelte sono buone i sistemi tendono a sostenere accoppiamento basso, coesione alta, facilità di comprensione, manutenzione ed estensione, possibilità di riuso*
- **Soluzione:** Assegnare una responsabilità alla classe che ha le informazioni necessarie per adempiere alla responsabilità.

Esempio

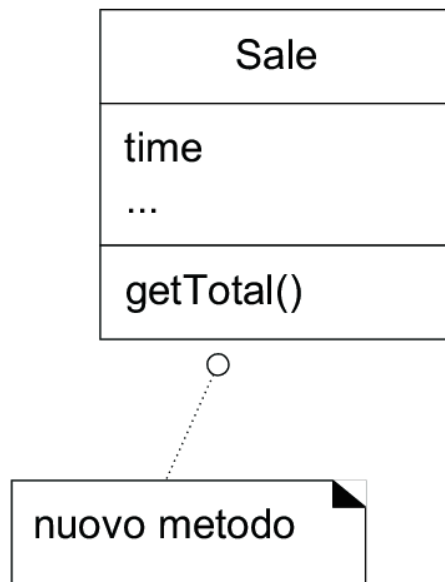


- Nell'applicazione NextGen POS, chi dovrebbe essere responsabile di conoscere il totale di una vendita?

Esempio

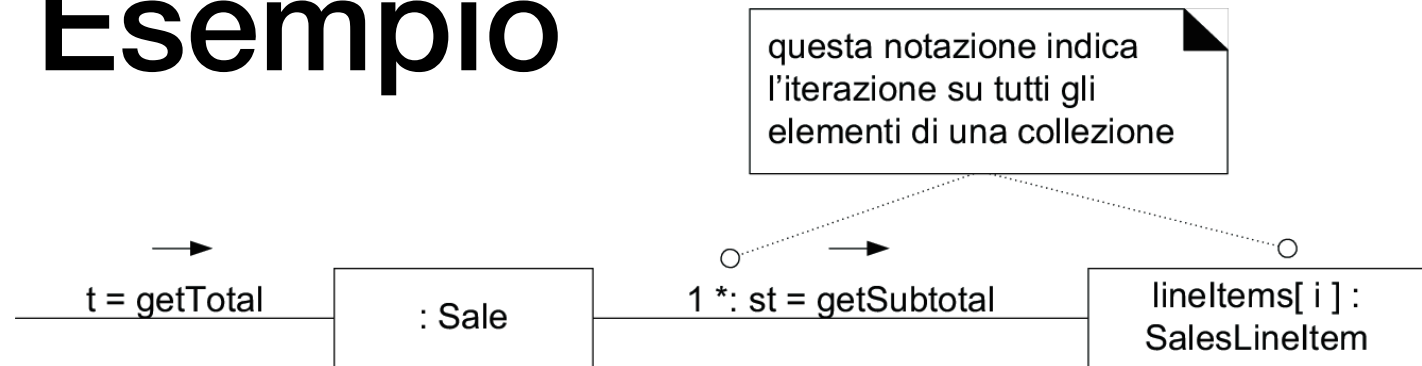


La classe concettuale Sale è un Information Expert per calcolare il totale di una vendita



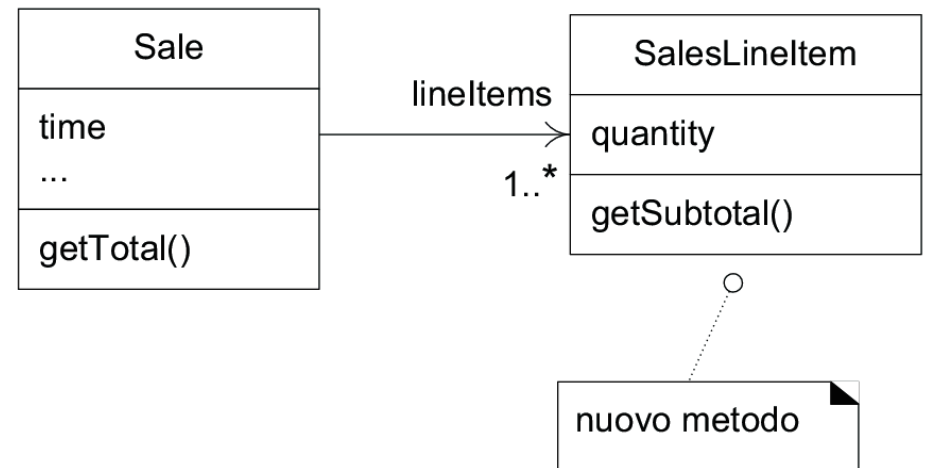
- aggiungo una classe software Sale al modello di progetto
- a questa classe assegno la responsabilità di conoscere/calcolare il suo total

Esempio

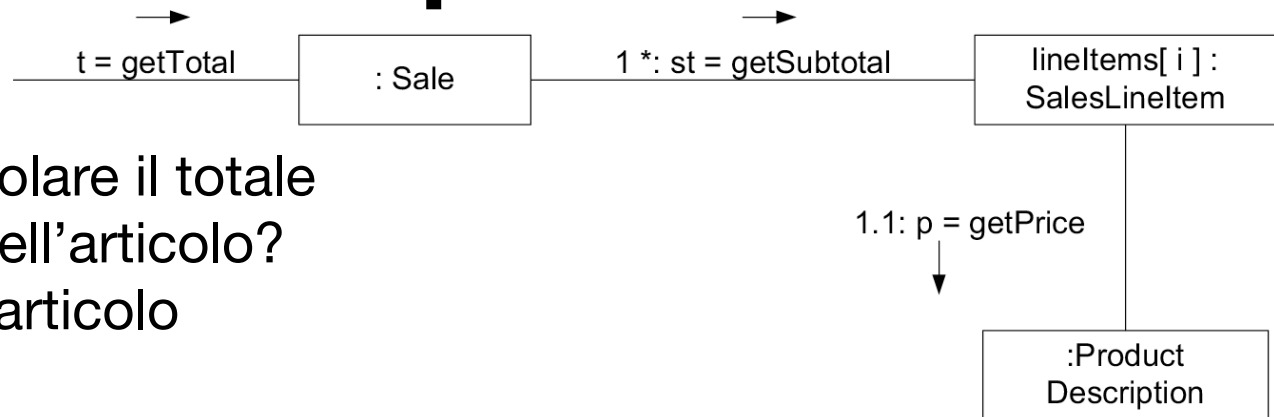


Come fa Sale a soddisfare la responsabilità che gli è stata assegnata?

- Sale può calcolare il totale della vendita come somma dei totali parziali delle righe di vendita degli articoli
- chi è responsabile di calcolare questi totali parziali?

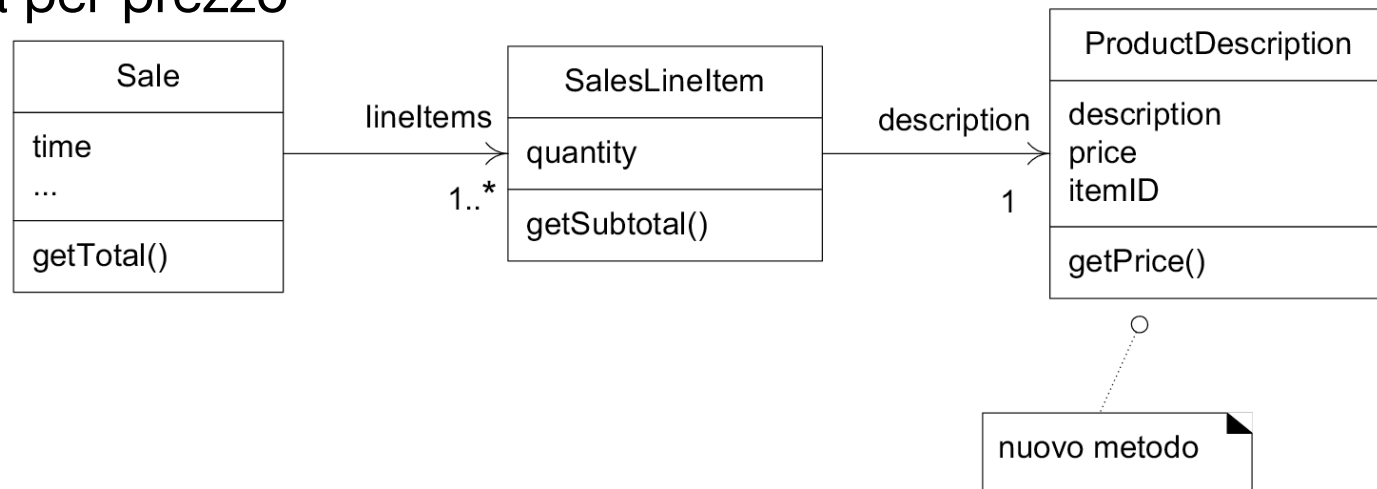


Esempio



Come fa SalesLinItem a calcolare il totale parziale della riga di vendita dell'articolo?
deve conoscere il prezzo dell'articolo

- SalesLinItem può calcolare il totale parziale moltiplicando quantità per prezzo



Esempio

Per assegnare la responsabilità di sapere e calcolare il totale di una vendita, sono state identificate e assegnate tre responsabilità

Classe di progetto	Responsabilità
<i>Sale</i>	sa calcolare il totale della vendita; conosce le righe di vendita della vendita
<i>SalesLineItem</i>	sa calcolare il totale parziale della riga di vendita; conosce il prodotto della riga di vendita
<i>ProductDescription</i>	conosce il prezzo del prodotto

Discussione

- Information Expert è un principio di base usato frequentemente nell'assegnazione di responsabilità
- per soddisfare una responsabilità è spesso necessario far collaborare molti esperti parziali delle informazioni
- Information Expert porta spesso a progetti in cui gli oggetti software fanno cose che normalmente vengono fatte sugli oggetti inanimati del mondo reale che essi rappresentano
- analogia con il mondo reale: normalmente si assegnano le responsabilità alle persone che posseggono le informazioni necessarie per svolgere un compito

Controindicazioni

- in alcuni casi, le soluzioni suggerite da Information Expert non sono opportune a causa di problemi di accoppiamento e di coesione
 - *Per esempio, chi deve essere responsabile di salvare una Sale in una base di dati?*
- un principio architetturale di base: progetta per una separazione dei principali interessi del sistema:
 - *Mettere logica applicativa in un luogo, mettere logica della base di dati in un altro luogo, e così via*

Vantaggi e pattern correlati

Vantaggi

- l'incapsulamento dell'informazione viene mantenuto
 - *solitamente favorisce un accoppiamento basso*
- il comportamento è distribuito tra tutte le classi che possiedono le informazioni richieste, incoraggiando la definizione di classi più coese e leggere
 - *favorisce una coesione alta*

Pattern o principi correlati

- Low Coupling e High Cohesion

Accoppiamento

- **L'accoppiamento** (coupling) è la misura di quanto un elemento è collegato, conosce o si basa su altri elementi.
- Una classe con un elevato grado di accoppiamento dipende da molte altre classi
- Classi con un alto accoppiamento comportano:
 - I cambiamenti in classi correlate obbligano a cambiamenti locali
 - Sono difficili da comprendere in isolamento
 - Sono difficili da riusare perché il loro riuso richiede anche il riuso delle classi da cui dipendono

Low Coupling

- **Problema:** come supportare una bassa dipendenza, un basso impatto dei cambiamenti e un maggiore riutilizzo?
- **Soluzione:** Assegnare una responsabilità in modo che l'accoppiamento rimanga basso.

Esempio

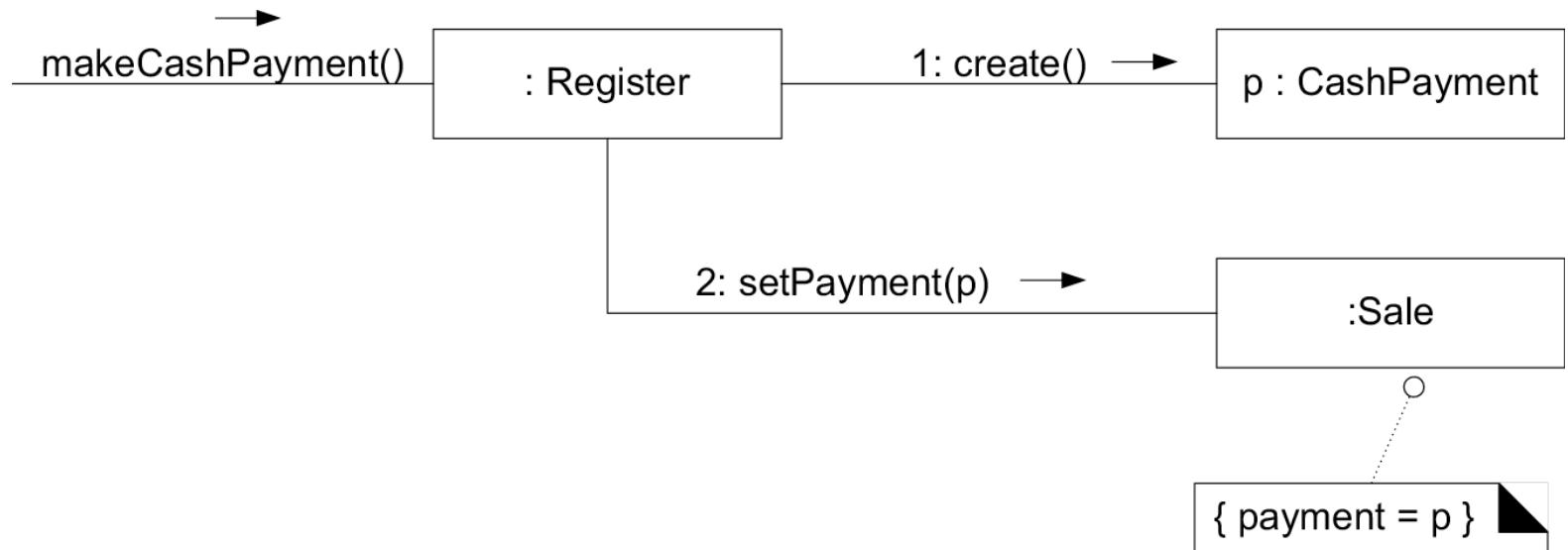
CashPayment

Register

Sale

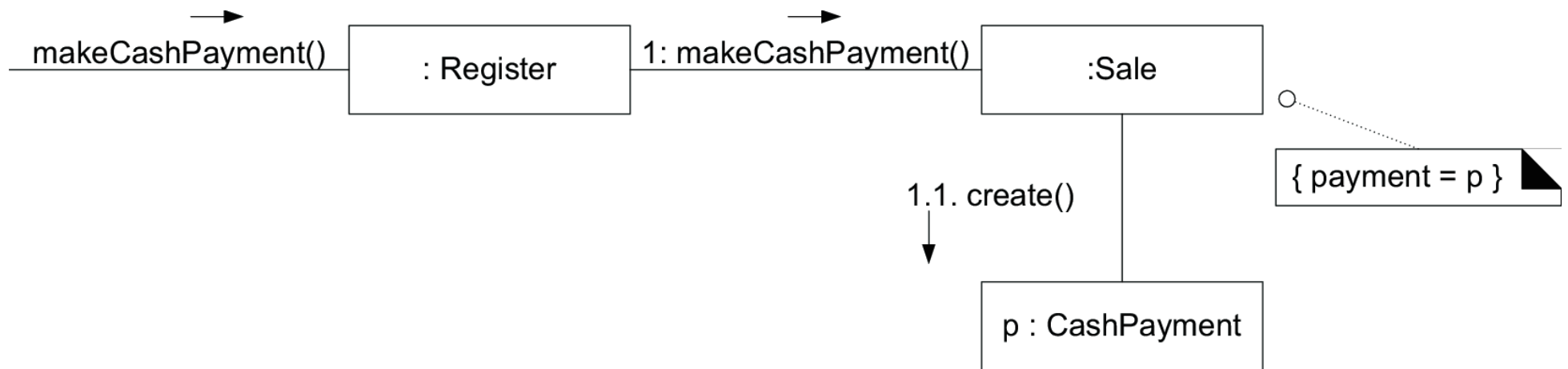
- Supponiamo di dover creare un'istanza di CashPayment e di associarla alla Sale.
- Quale classe dovrebbe essere responsabile di ciò?
- Poiché un Register nel dominio del mondo reale registra i pagamenti, il pattern Creator suggerisce Register come candidato.

Esempio



- Register potrebbe quindi inviare un messaggio `setPayment` a `Sale`, passando il nuovo `CashPayment` come parametro.
- L'assegnazione delle responsabilità accoppia la classe `Register` alla conoscenza della classe `CashPayment`.

Esempio



- Una soluzione alternativa è quella in cui Sale crea CashPayment
- Nessun accoppiamento tra il Register e il CashPayment.

Discussione

- Low Coupling è un principio di valutazione, da tenere a mente durante tutte le decisioni di progetto
- Nei linguaggi OO, le forme di accoppiamento da un tipo X a un tipo Y comprendono
 - *la classe X ha un attributo di tipo Y o riferenzia una collezione di oggetti di tipo Y*
 - *un oggetto di tipo X richiama servizi o operazioni di un oggetto di tipo Y*
 - *Un oggetto di tipo X crea un oggetto di tipo Y*
 - *Il tipo X ha un metodo che contiene un elemento (parametro, variabile locale, tipo di ritorno) di tipo y*
 - *X è una sottoclasse (diretta o indiretta) di Y*
 - *Y è una interfaccia, e X implementa Y*

Discussione

- Low Coupling incoraggia l'assegnazione di responsabilità in modo tale che l'accoppiamento raggiunto non sia “troppo alto”
 - *Ci sarà sempre un certo accoppiamento tra gli oggetti, altrimenti non ci sarebbe collaborazione.*
- Low Coupling sostiene la progettazione di classi indipendenti, con un basso impatto dei cambiamenti
- una sottoclasse è accoppiata fortemente alla sua superclasse
 - *valuta meccanismi alternativi all'estensione di classi (composizione e delega)*

Controindicazioni, vantaggi e pattern correlati

Controindicazioni

- l'accoppiamento con elementi stabili non è un problema
- il problema è l'accoppiamento alto con elementi instabili

Vantaggi

- il cambiamento di altre classi ha un impatto basso
- consente di definire classi semplici da comprendere in isolamento e convenienti da riusare

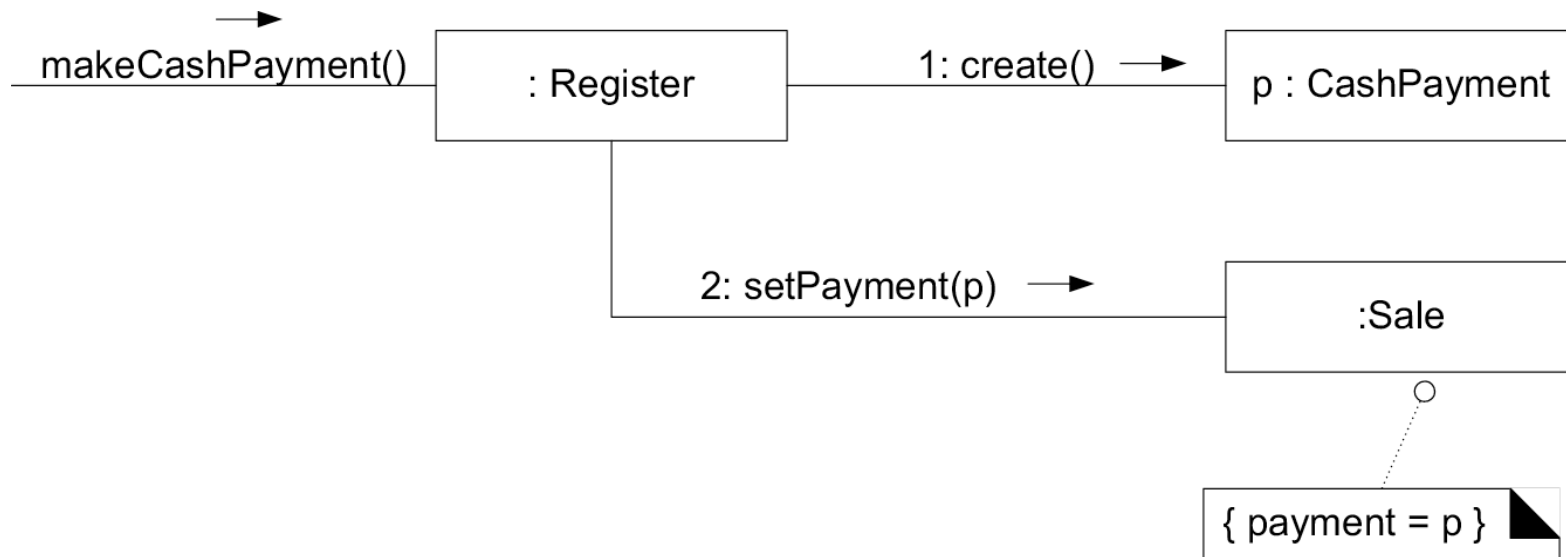
Pattern correlati

- Protected Variations

High Cohesion

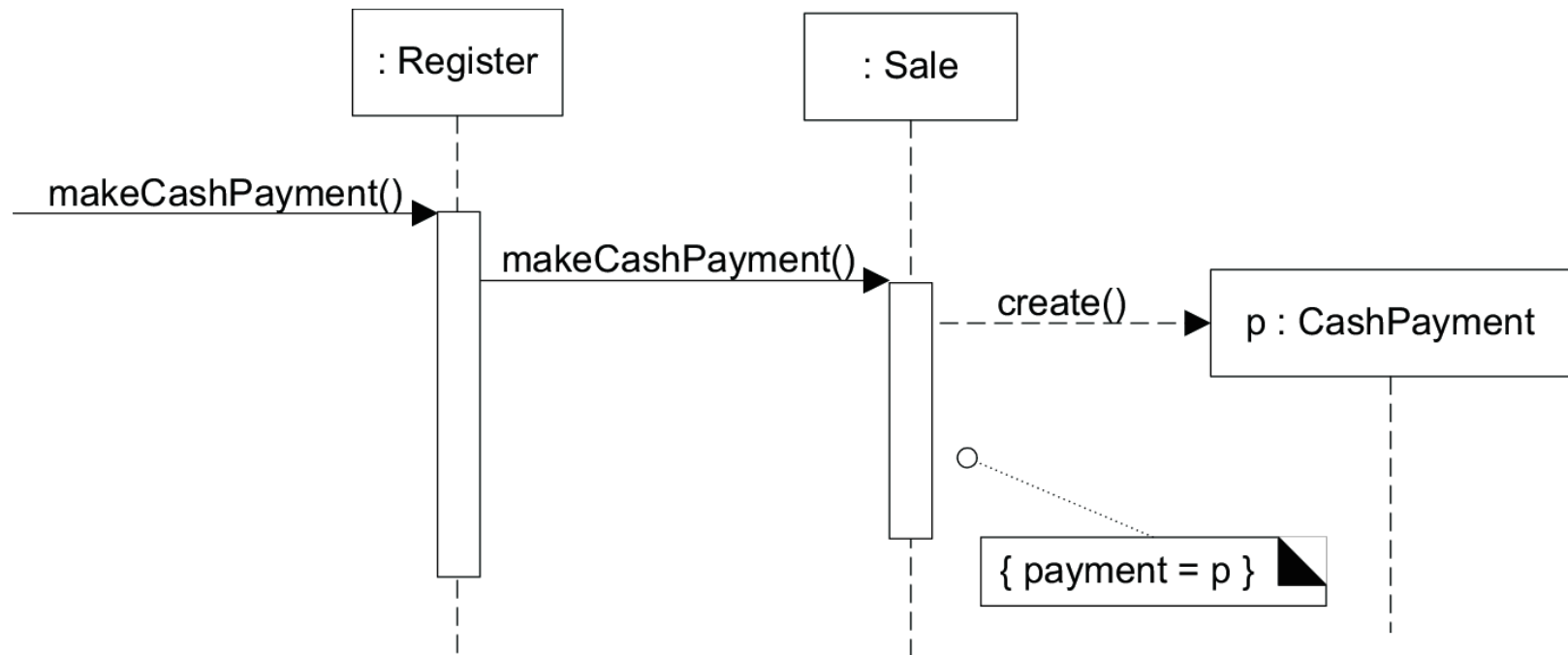
- **Problema:** come mantenere gli oggetti focalizzati, comprensibili e gestibili e, come effetto collaterale, sostenere Low Coupling?
- La **coesione** (coesione funzionale) è una misura di quanto fortemente siano correlate e concentrate le responsabilità di un elemento
 - *Un classe con responsabilità altamente correlate che non esegue una quantità di lavoro eccessiva ha una coesione alta*
- **Soluzione:** assegna una responsabilità in modo che la coesione rimanga alta

Esempio



- Supponiamo di dover creare un'istanza di `CashPayment` e di associarla a `Sale`. Quale classe dovrebbe essere responsabile di questo?
- Per Creator, `Register` è un candidato.
- Se si fa l'ipotesi che `Register` sia il controller per il caso d'uso in cui deve avvenire la creazione e il pagamento per la vendita, `Register` assume non solo la responsabilità di ricevere operazione sistema `makeCashPayment` ma anche parte della responsabilità di soddisfarla

Esempio



- Soluzione alternativa
 - coesione maggiore e accoppiamento minore

Discussione

High Cohesion, come Low Coupling, è **un principio di valutazione**, da tenere in considerazione durante tutto il progetto. Ci sono diversi livelli di coesione:

- Coesione molto bassa: una classe è la sola responsabile di molte cose in aree funzionali molto diverse
 - *classe RDB-RPC-Interface che è responsabile dell'interazione con DB relazionale e chiamate remote, 2 aree funzionali molto diverse*
- Coesione bassa: una classe ha da sola la responsabilità di un compito complesso in una sola area funzionale
 - *classe RDBInterface per accesso con DB relazionale, la classe può avere centinaia o migliaia di metodi*
- Coesione moderata: una classe ha da sola responsabilità leggere in poche aree diverse, che sono logicamente correlate al concetto rappresentato dalla classe, ma non l'una con l'altra
 - *Company completamente responsabile di conoscere i suoi dipendenti e conoscere le proprie informazioni finanziarie, aree correlate alla company, ma non tra loro*
- Coesione alta: una classe ha responsabilità moderate in un'unica area funzionale e collabora con le altre per svolgere i suoi compiti
 - *classe RDBInterface responsabile per l'interazione con le DB relazionali, essa interagisce con dozzine di altre classi per recupero e salvataggio degli oggetti*

Discussione

- Come regola pratica, una classe con coesione alta ha pochi metodi, con delle funzionalità altamente correlate e focalizzate e fa poco lavoro
- Analogia con il mondo reale: i manager che non hanno imparato a delegare soffrono di una coesione bassa, e sono pronte “a perdere il controllo di se”

Controindicazioni, Vantaggi

Controindicazioni

- casi in cui la coesione bassa è giustificabile
 - *raggruppamento di responsabilità o codice in una classe o componente gestito e mantenuto da una singola persona*
 - *per motivi di efficienza, può essere necessario definire pochi oggetti server distribuiti che forniscono servizi ampi e poco coesi*

Vantaggi

- chiarezza e facilità di comprensione
- manutenzione ed evoluzione semplificata
- spesso favorisce un accoppiamento basso
- funzionalità altamente correlate a grana fine favoriscono il riuso, perché una classe coesa può essere usata per uno scopo specifico

Pattern correlati

- Il pattern Single-Responsibility Principle

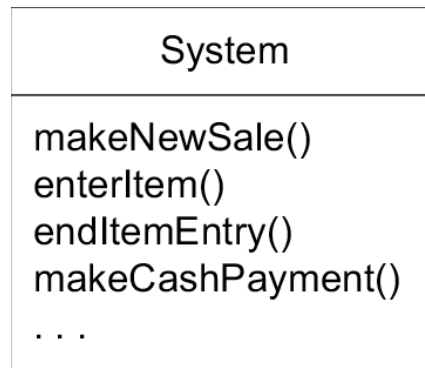
Controller

- **Problema:** quale è il primo oggetto oltre lo strato UI che deve ricevere e coordinare (controllare) l'esecuzione di una operazione di sistema?
- Le **operazioni di sistema** sono gli eventi di input principali nel sistema
- Un **controller** è il primo oggetto oltre lo strato di UI che è responsabile di ricevere e gestire un messaggio di un'operazione di sistema

Controller

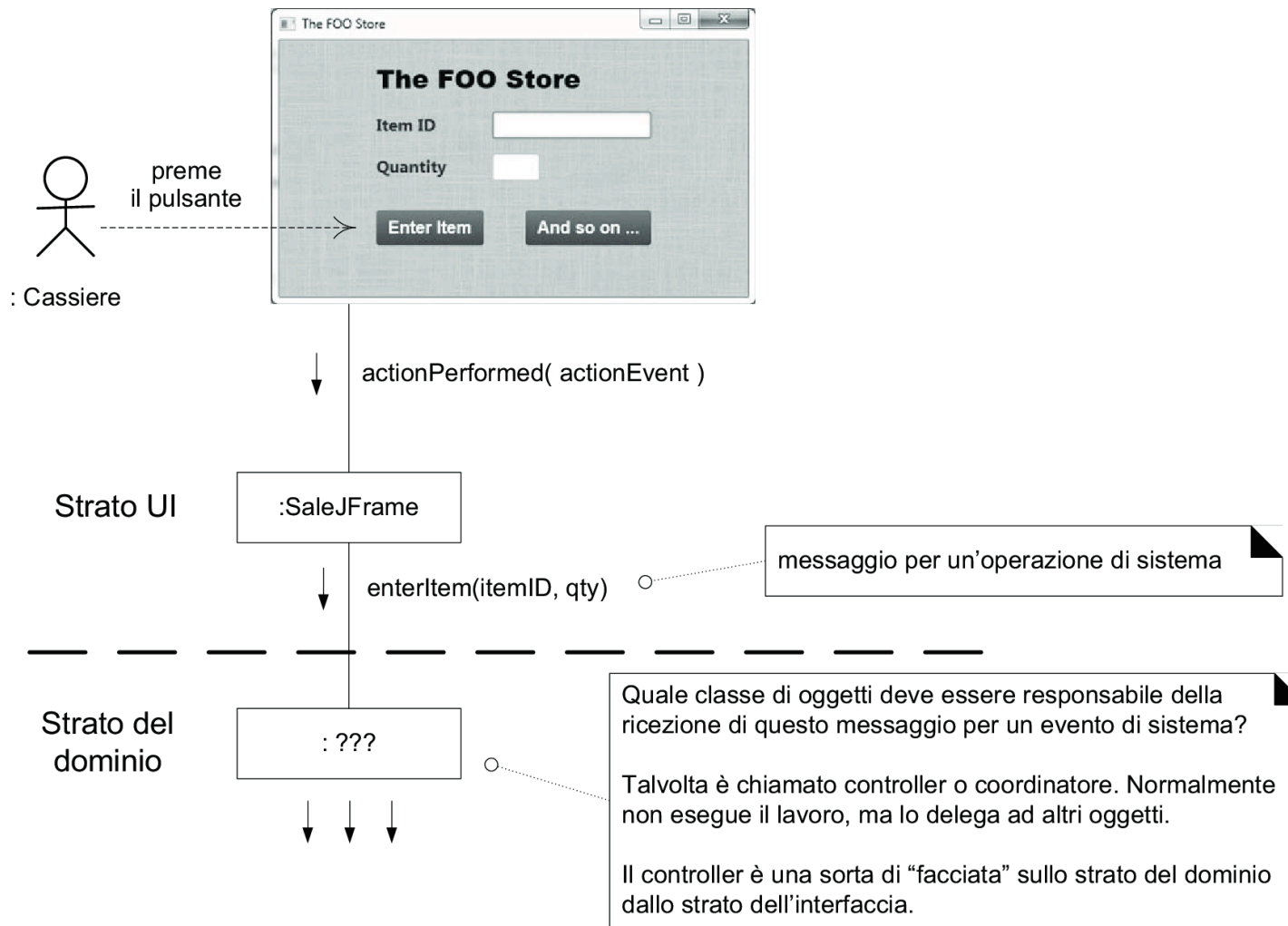
- **Soluzione:** Assegna la responsabilità a un oggetto che rappresenta una delle seguenti scelte :
 - *Rappresenta il «sistema» complessivo, un «oggetto radice», un dispositivo all'interno del quale viene eseguito il software, un punto di accesso al software o un sottosistema principale (Façade Controller)*
 - *Rappresenta uno scenario di un caso d'uso all'interno del quale si verifica l'operazione di sistema (Use Case Controller)*

Esempio



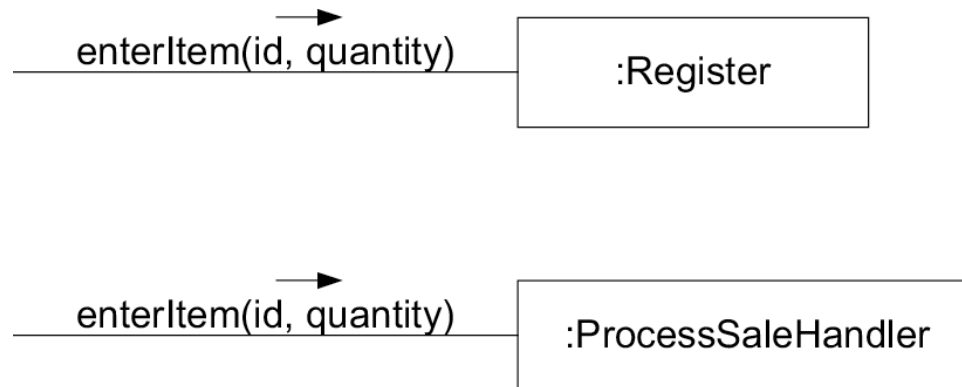
- Durante l'analisi, le operazioni di sistema vengono assegnata a una classe System
 - *questo non significa che bisogna utilizzare una classe software di nome System*

Esempio



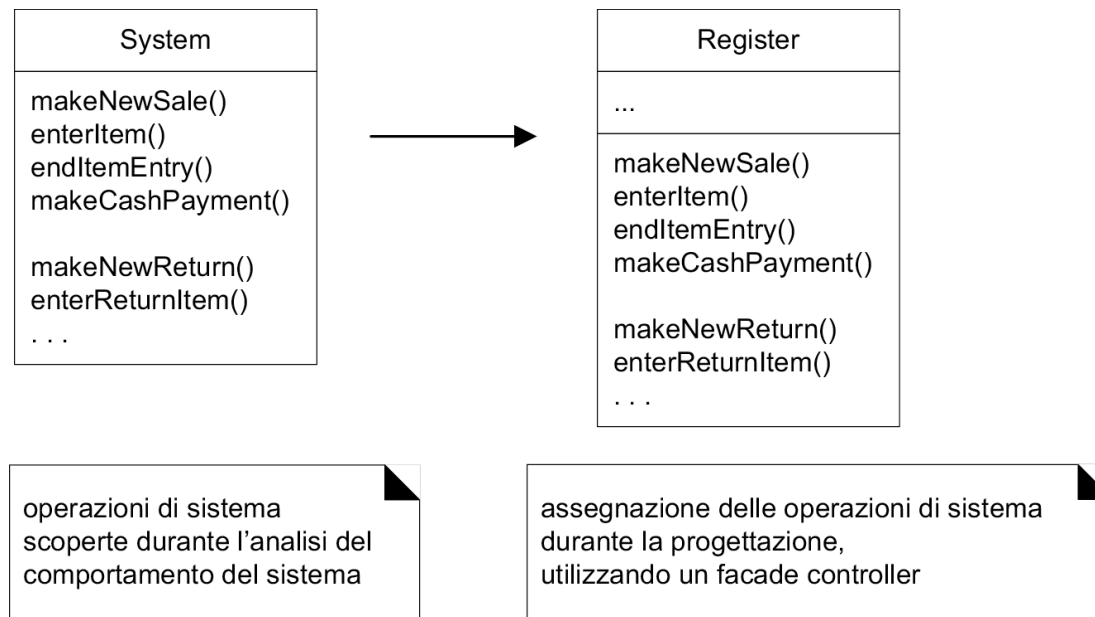
Esempio

- Scelte suggerite da controller



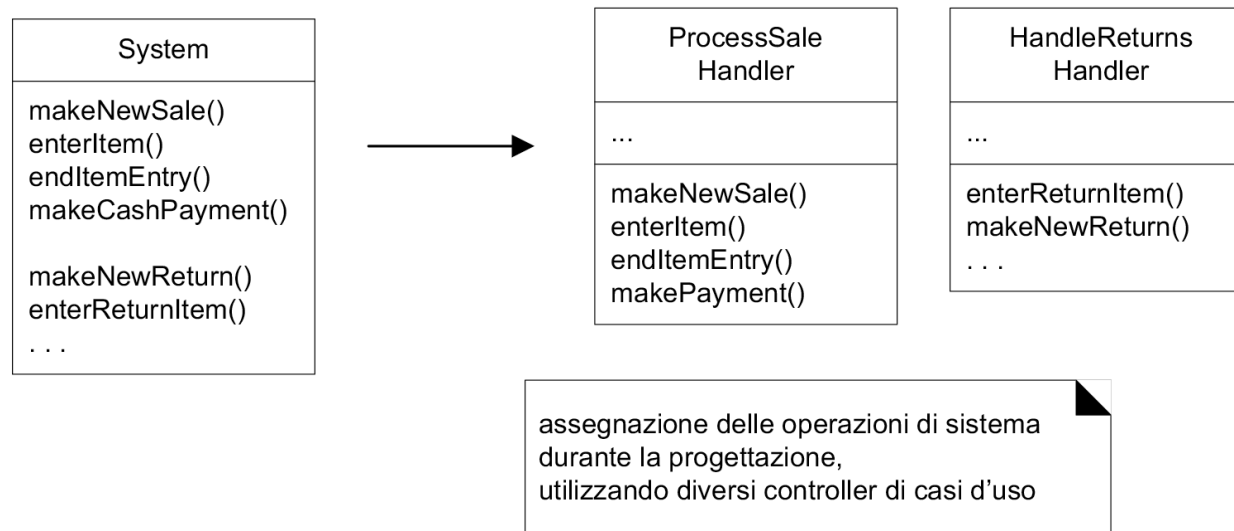
Esempio

- Un facade controller



Esempio

- Più use-case controller



Implementazione Controller (Java Swing)

```
package com.craiglarman.nextgen.ui.swing;

...

/* Finestra per Elaborare vendita. */

public class ProcessSaleJFrame extends JFrame {

    /* mantiene un riferimento al controller */

    private Register register;

    /* Crea un nuovo ProcessSaleJFrame, il cui controller
    è register . */

    public ProcessSaleJFrame(Register register) {

        this.register = register;

        ...

    }

    /* Bottone per enterItem. */

    private JButton BTN_ENTER_ITEM;
```

```
/* Gestione del bottone per enterItem. */

private JButton getBTN_ENTER_ITEM() {

    /* se esiste, restituiscilo */

    if (BTN_ENTER_ITEM!=null) return BTN_ENTER_ITEM;

    /* non esiste, crealo */

    BTN_ENTER_ITEM = new JButton("Enter item");

    /* definisci un ascoltatore per il bottone */

    BTN_ENTER_ITEM.addActionListener( new ActionListener() {

        public void actionPerformed( ActionEvent e) {

            /* Transformer trasforma da stringhe a tipi opportuni */

            ItemID id = Transformer toItemID(getTXT_ID().getText ());

            int qty= Transformer.toInt(getTXT_QTY().getText ());

            /* DELEGA L'ESECUZIONE AL CONTROLLER */

            register.enterItem( id,qty);

        }

    });

    return BTN_ENTER_ITEM;

}

...

}
```

Discussione

- È semplicemente un pattern di delega – da UI a dominio
- un controller deve delegare ad altri oggetti il lavoro da fare durante l'operazione di sistema, limitandosi a controllare o coordinare le attività
- un difetto comune nella progettazione dei controller consiste nel dare troppe responsabilità a un controller (Controller gonfi)
- facade controller o use-case controller?
- uno use-case controller è una Pure Fabrication

Vantaggi

- aumenta la possibilità di riuso dello strato del dominio e consente di definire interfacce utente “insreribili”
- Opportunità di ragionare sullo stato dei casi d’uso
 - *assicurarsi che le operazioni sistema si susseguono in una sequenza legale*
 - *ragionare sullo stato corrente delle attività e dell'operazioni all'interno di un caso di uso in corso di esecuzione*

Problemi e soluzioni

un **controller gonfio** è un controller con coesione bassa che gestisce responsabilità in troppi contesti

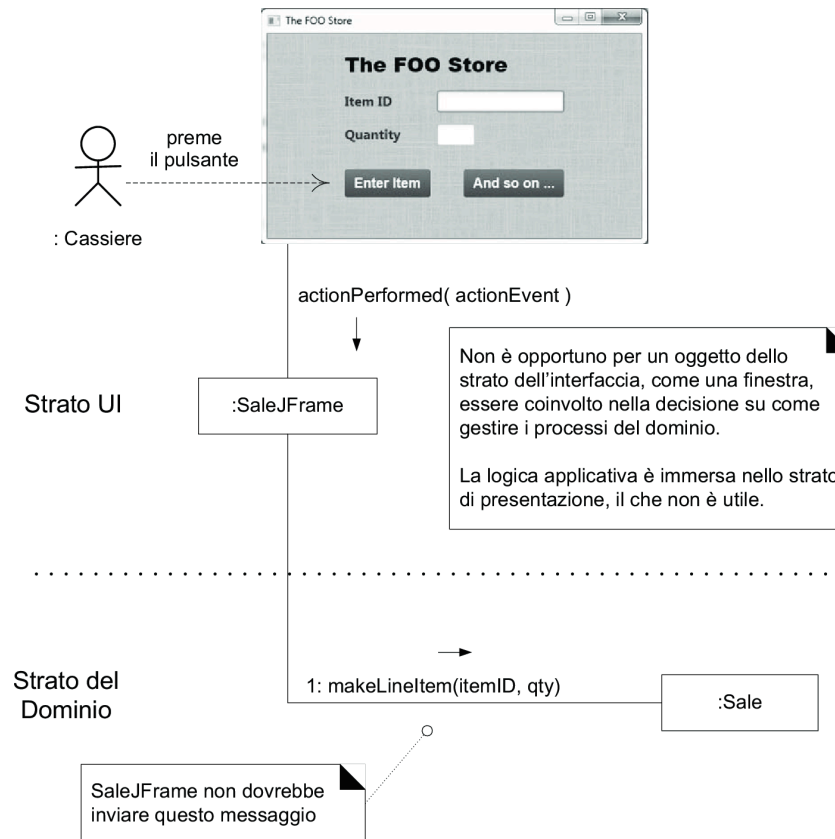
- esiste una unica classe controller che riceve tutti i numerosi eventi di sistema
- Il controller svolge parte del lavoro prima di delegarlo
- Il controller ha numerosi attributi e conserva informazioni sul sistema e sul dominio

Soluzioni

- Progettare il controller in modo che deleghi la gestione delle operazioni di sistema
- Aggiungere altri controller

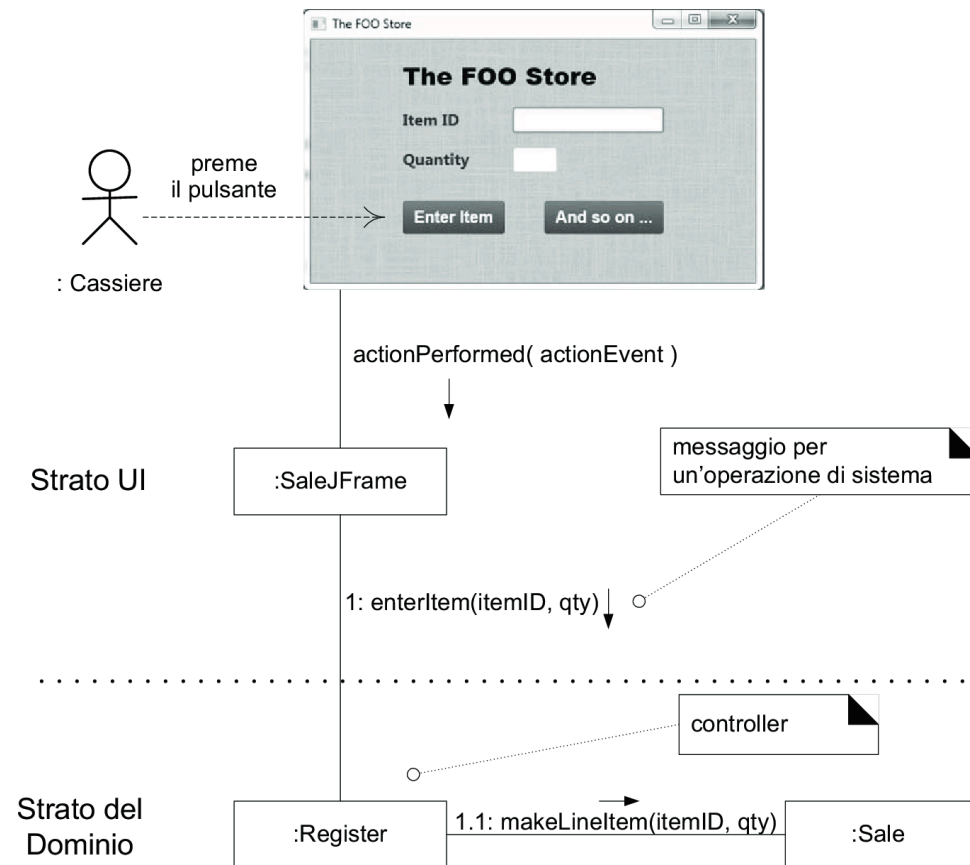
Problemi e soluzioni

Lo strato UI non gestisce gli eventi di sistema



Problemi e soluzioni

Lo strato UI non gestisce gli eventi di sistema



Pattern correlati

- In un sistema di gestione dei messaggi, ogni messaggio può essere rappresentato e gestito con un oggetto **Command** separato
- Un facade controller è un tipo **Facade**
- Porre la logica di dominio nello strato del dominio e tenerla indipendente dallo strato della presentazione fa parte del pattern **Layers**
- Un controller di caso d'uso è una specie di **Pure Fabrication**